

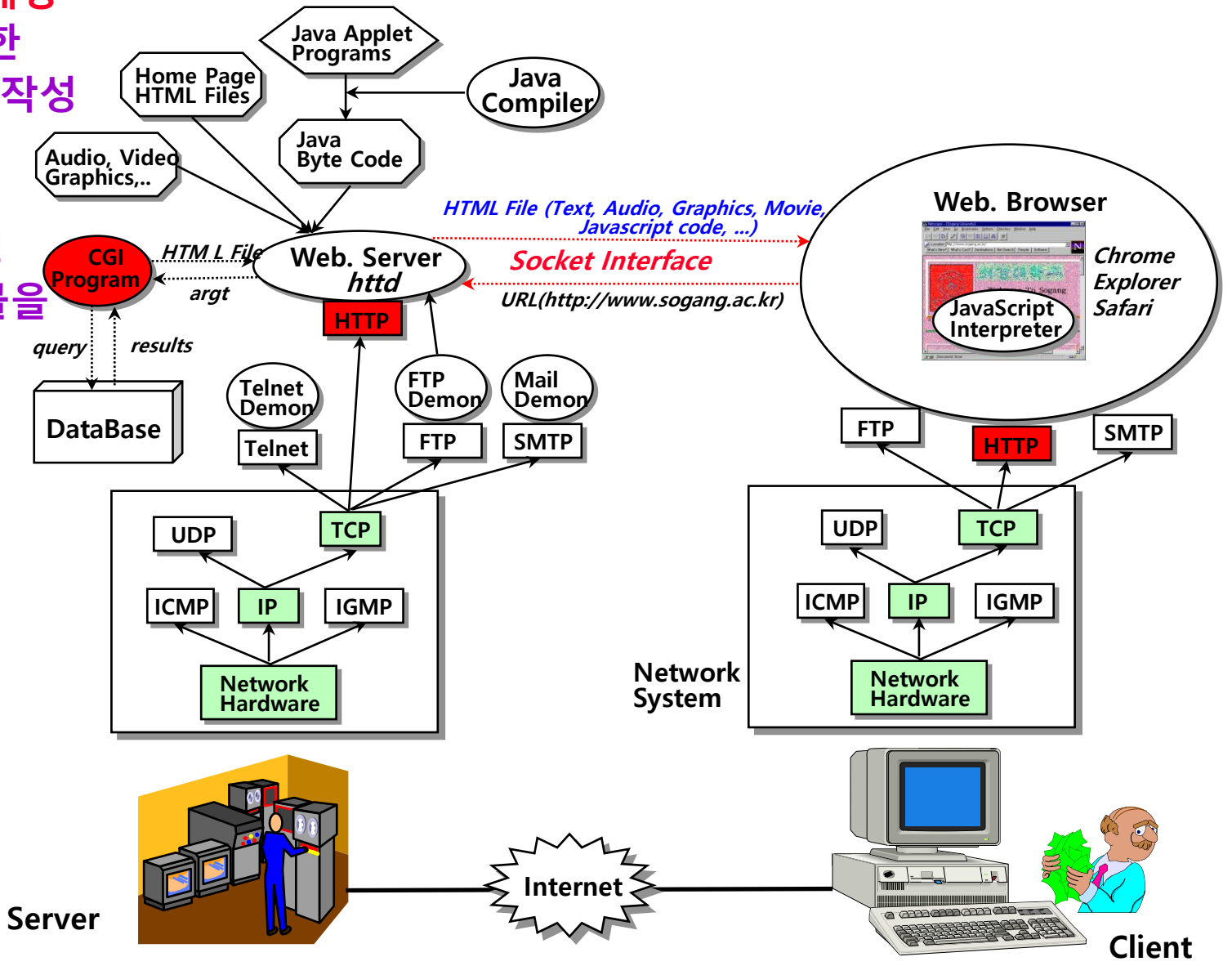
HTML/HTTP 및 CGI 프로그래밍

1. 웹서버 프로그래밍 소개
2. HTML을 이용한 문서 작성
3. HTTP 프로토콜
4. CGI 프로그래밍
5. 프로그래밍 과제

서강대학교
Sogang University

1. Web 서버 프로그래밍 소개

- Web 서버 프로그래밍
  - HTML을 이용한 Home 페이지 작성
  - CGI를 이용한 Web. 서버 프로그램 작성
  - HTTP 프로토콜을 이용한 탐색



2. HTML을 이용한 문서 작성

- HTML (Hypertext Markup Language)이란 ?
  - Hypertext 문장을 만들기 위하여 쓰이는 간단한 markup 언어
    ⇔ (참고) Markup language : 문서 중간에 문자나 문장의 형태에 대한 명령어를 삽입하는 형태의 문서 표현 언어 (예: troff, Latex 등)
  - 1990년 이후 Web.에서 사용되는 문서 양식
  - SGML (Standard Generalized Markup Language)의 application
  - HTML 1.0/2.0/3.0
    ⇔ 1.0 / 1991 (Tim Berners Lee)
    ⇒ Web.에서 사용되는 기본적인 태그에 대한 정의
    ⇔ 2.0 / 1994 (Dan Commelly)
    ⇒ 양식의 지원
    ⇔ 3.0
    ⇒ 테이블, 수학식, 개선된 양식 (Fill-out Form)
    ⇒ 문서에 특정한 도구바/배너 영역, 그림 삽입, style-sheet의 첨부
- HTML 문서를 작성하는 방법
  - 보통 문서 편집기 (예: vi, emacs, ...)를 이용하는 방법
  - HTML 편집기 (WYSIWYG 형태)를 사용하는 방법 (Windows95用, X-Windows用, ..)
  - 이미 작성되어 있는 어떤 형태의 문서를 HTML 문서로 바꾸는 방법
  - (tex2html, ps2html, c++2html, txt2html, hypermail, man2html, rtf2html, ...)

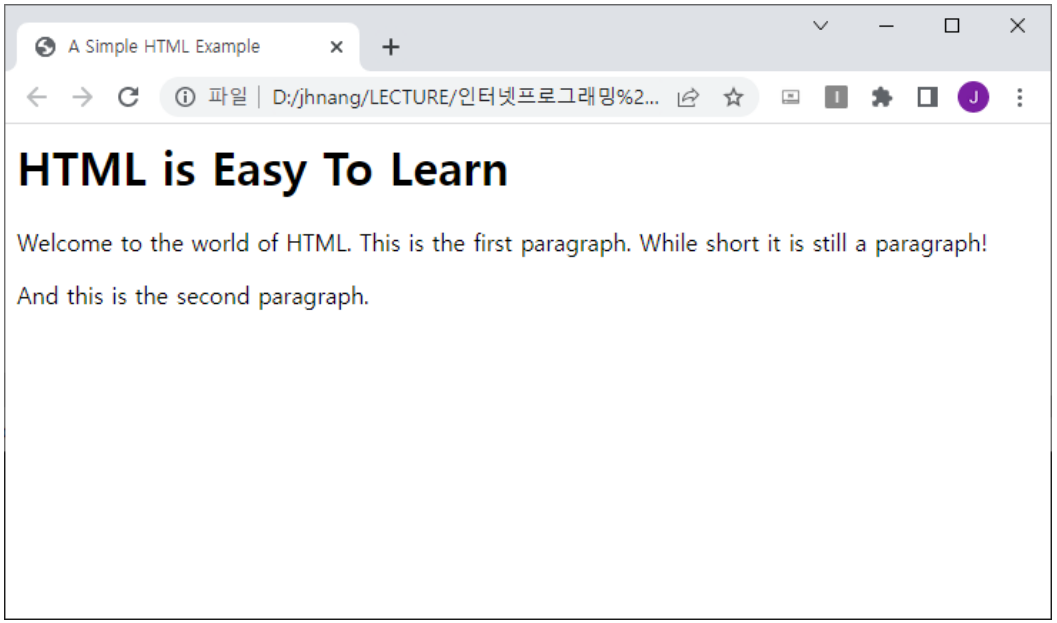
- **Basic Document Structure**
  - HTML text file in a publicly readable directory
    - ⇔ the HTTP server sends it over to the client for imaging
  - HTML defines the basic structure of a document, but the exact appearance is determined by the browser dynamically
  - Text and tags
    - ⇔ tags are enclosed in <>'s
    - ⇔ some tags occurs by themselves e.g.
, others are paired to enclose some text <h1> .... </h1>
    - ⇔ upper and lower case tags act the same, e.g., <H1> .... </h1>
    - ⇔ "whitespace" is ignored
  - some elements may include an attribute, which is additional information that is included inside the start tag
  - Not all tags are supported by all WWW browsers
    - ⇔ if a browser does not support a tag, it (usually) just ignores it

- **The Minimal HTML Document**
  - Every HTML document should contain certain standard HTML tags
  - Each document consists of head and body text
    - ⇔ head : contains the title
    - ⇔ body : contains the actual text that is made up of paragraphs, lists, and other elements
  - Example 1

HTML Source

```
<html>
 <head>
  <TITLE>A Simple HTML Example</TITLE>
 </head>
 <body>
  <H1>HTML is Easy To Learn</H1>
    <P>Welcome to the world of HTML.
       This is the first paragraph. While
       short it is still a paragraph!</P>
   <P>And this is the second paragraph.</P>
 </body>
</html>
```

Browser Output



• Example 2 : HTML Source

```
<HTML>
 <HEAD>
<TITLE>NCSA Beginner's Guide--A Longer Example</TITLE>
</HEAD>
<BODY>

<H1>Something a Bit More Complex</H1>
<P>
This is a relatively simple HTML document.  Simple
is in the eye of the beholder, but if you study what is included
in this beginner's guide, you can create documents like this
with no problem!
</P>

<H2>Special Effects (header 2)</H2>
<P>
The second paragraph shows some special effects: a
word in <I>italics</I> and a word in <B>bold</B>.
Here is an inlined GIF image: <IMG SRC="BarHotlist.gif">.
</P>
<P>
This is the third paragraph, which demonstrates links.
A hypertext link
from the word <A HREF = "Staff/TracyHome.html">me!</A>
to a document called "Staff/TracyHome.html" exists but if you
try to follow this link, you will get an error screen.
</P>
```

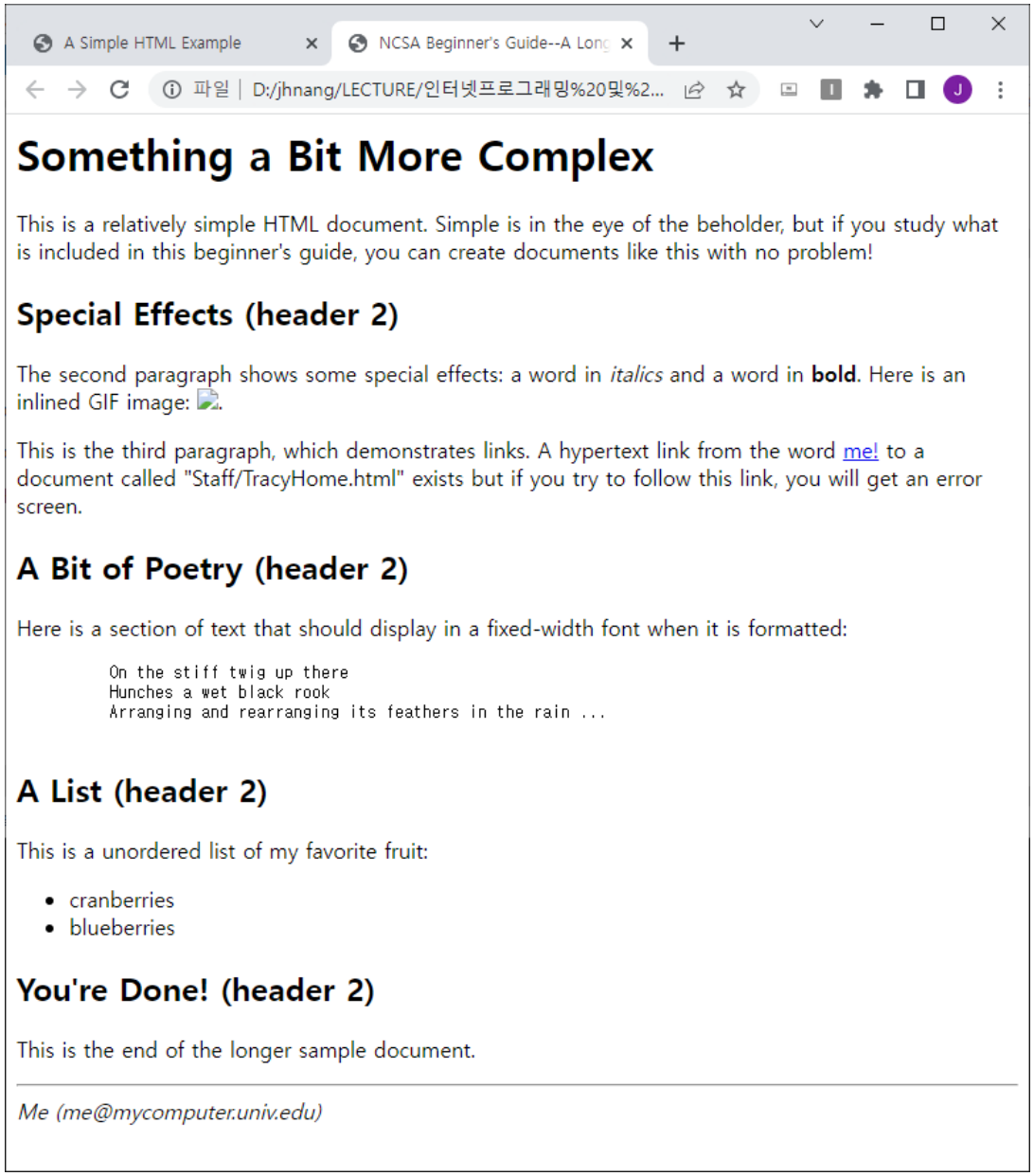
```
<H2>A Bit of Poetry (header 2)</H2>
<P>
Here is a section of text that should display in a
fixed-width font when it is formatted:
</P>
  <PRE>
     On the stiff twig up there
     Hunches a wet black rook
     Arranging and rearranging its feathers in the rain ...
  </PRE>

<H2>A List (header 2)</H2>
<P>
This is a unordered list of my favorite fruit:
</P>
  <UL>
  <LI> cranberries
  <LI> blueberries
  </UL>

<H2>You're Done! (header 2)</H2>
<P>
This is the end of the longer sample document.
</P>

<HR>
<ADDRESS>Me (me@mycomputer.univ.edu)</ADDRESS>
</BODY>
<HTML>
```

- **Example 2 : Browser Output 1**



2.1 Markup Tags

- **Basic Tags**
  - **<html> ... </html>**
    - ⇔ encloses the entire HTML source
  - **<head> .... </head>**
    - ⇔ enclose the header section of an HTML
  - **<title> .... </title>**
    - ⇔ the title phrase for this page
    - ⇔ the title is displayed somewhere on the browser window
    - ⇔ used by search engine
  - **<body> .... </body>**
    - ⇔ the body is basically all of the document but the header
  - **<h1> Larger Header </h1>**
  - **<h2> Smaller Header </h2>**
  - ** **
    - ⇔ Marks the beginning of a block of text which should stick together as single paragraphs
      - ⇒ *All of the text* after until another paragraph or header marker will be gather up into a single paragraph
    - ⇔ the can be omitted
    - ⇔ ** **

```
<HTML>
  <HEAD>
    <TITLE>
      인터넷 프로그래밍 강의
    </TITLE>
  </HEAD>
  <BODY>
    <H1> TCP/IP </H1>
    <P> TCP/IP는 인터넷 ... </P>
    <P> HTML은 우리가  ... </P>
  </BODY>
</HTML>
```

- **Lists**
  - **Unnumbered List : <UL> .... </UL>**

```
<UL>
<LI> apples
<LI> bananas
<LI> orange
</UL>
```

- apples
- bananas
- orange

  - **Numbered List : <OL> .... </OL>**

```
<OL>
<LI> apples
<LI> bananas
<LI> orange
</OL>
```

1. apples
2. bananas
3. orange

  - **Definition List : <DL> .... </DL>**

```
<DL>
<DT> NCSA
<DD> NCSA, the National Center for Supercomputing
Applications, is located on the campus of the ...
<DT> Cornell Theory Center
<DD> CTC is located on the campus of Cornell Univ.
in the Ithaca, New York.
</DL>
```

NCSA
: NCSA, the National Center for Supercomputing Applications, is located on the campus of the ...

Cornell Theory Center
: CTC is located on the campus of Cornell Univ. in the Ithaca, New York.

  - **Nested List**

```
<UL>
<LI> 공과 대학
      <UL>
      <LI> 전자계산학과
      <LI> 전자공학과
      <LI> 화학공학과
      <LI> 기계공학과
      <UL>
<LI> 이과대학
      <UL>
      <LI> 수학과
      <LI> 물리학과
      <LI> 화학과
      <LI> 생명과학과
      </UL>
</UL>
```

- **Preformatted List** : **<PRE> ..... </PRE>**
  - use the <PRE> tag to generated text in a fixed-width font
  - this tag also makes spaces, new lines, and tabs significant
  - useful for program listing

```
<PRE>
#!/bin/csh
cd $SCR
cfs get mysrc.f:mycfsdir/mysrc.f
cfs get myinfile:mycfsdir/myinfile
fc -02 -o mya.out mysrc.f
my a.out
cfs save myoutfile:mycfsdir/myoutfile
rm *
</PRE>
```

```
#!/bin/csh
cd $SCR
cfs get mysrc.f:mycfsdir/mysrc.f
cfs get myinfile:mycfsdir/myinfile
fc -02 -o mya.out mysrc.f
my a.out
cfs save myoutfile:mycfsdir/myoutfile
rm *
```

- **Forced Line Breaks** : **
**
  - forces a line break with no extra (white) space between lines

- **Horizontal Rules** : **<HR>**
  - produce a horizontal line
  - useful to separate section of document

```
<HR SIZE=4  ALIGN=right  WIDTH=50%>
```

- **Address** : <ADDRESS> .... </ADDRESS>
  - the <ADDRESS> tag is generally used to specify the author of a document, a way to contact the author, and a revision date

```
<ADDRESS>
인터넷 프로그래밍 및 응용 / 서강대학교 전산과 / jhnang@ccs.sogang.ac.kr / revised September 1996
</ADDRESS>
```

인터넷 프로그래밍 및 응용 / 서강대학교 전산과 / jhnang@ccs.sogang.ac.kr / revised September 1996

- **Extended Quotations** : <BLOCKQUOTE> ..... </BLOCKQUOTE>
  - to include lengthy quotations in a separate block on the screen
  - most browsers generally change the margins for the quotation to separate it from surrounding text

```
lala <BR>
<BLOCKQUOTE>
   <P>Omit needless words.</P>
   <P>Vigorous writing is concise. A sentence
        should contain no unnecessary words </P>
   --William Strunk, Jr., 1918
   </BLOCKQUOTE>
```

lala

Omit needless words.

Vigorous writing is concise. A sentence should contain no unnecessary words,

--William Strunk, Jr., 1918

- **Character Formatting**
  - **Logical vs. Physical Styles**
    - ⇔ Why ?
      - ⇒ in SGML, content is divorced from presentation
    - ⇔ Logical Styles
      - ⇒ : 강조 , : 강한 강조, <code> : 컴퓨터 코드, <cite> : 인용구
      - ⇒ <kbd> : 키보드 키 표시, <samp> : sample output, <var> : 변수
    - ⇔ Physical Style
      - ⇒ <b> : bold, <i> : italic, <u> : underline, <tt> : typewriter font
  - **Escape Sequence**
    - ⇔ escape sequence
      - ⇒ &lt : '<' , &gt : '>', &amp : '&'
    - ⇔ special character
      - ⇒ &ouml, &ntilde, &Egrave
  - **Blinking** : <BLINK> .... </BLINK>
- **Comment**
  - <!--text-->

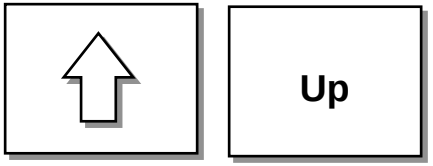
- **Linking**
  - link text and/or an image to another document or section of a document
  - a browser highlights the identified text or image with color and/or underlines to indicate that it is a hyperlink
  - tag for hyperlink : `<A HREF="filename"> .... </A>`
    - ⇔ `<A HREF="Sogang.html"> Sogang </A>`
    - ⇔ `<A HREF="../Sogang.html"> Sogang </A>`
    - ⇔ `<A HREF="/tmp/Sogang.html"> Sogang </A>`
  - WWW uses URL to specify the location of files on other servers
    - ⇔ `scheme://host.domain [:port]/path/filename`
      - ⇒ `scheme`
        - → `file` : a file on local system
        - → `ftp` : a file on an anonymous FTP server
        - → `http`: a file on a WWW server
        - → `gopher` : a file on a Gopher server
        - → `WAIS` : a file on WAIS server
        - → `news` : a Usenet newsgroup
        - → `telnet` : a connection to a Telnet-based service

NCSA's Beginner's Guide to HTML

    - ⇔ Example :

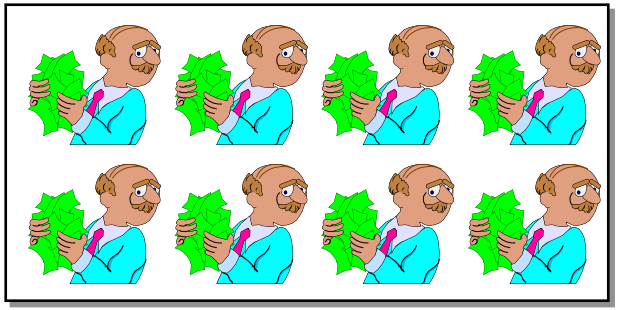
```
<A HREF="http://www.ncsa.uiuc.edu:/General/Internet/WWW/HTMLPrimer.html">
NCSA's Beginner's Guide to HTML </A>
```

- **Alternative Text for Images**
  - ⇔ for browsers that can't display images
  - ⇔ `<IMG SRC="UpArrow.gif" ALT="Up">`



- **Background Graphics**
  - ⇔ newer version of Web browsers can load an image and use it as a background when displaying a page
  - ⇔ a browsers takes the image and repeats it across and down to fill the browser windows (tiling)

```
<BODY BACKGROUND="filename.gif">
```



- **Background Color**
  - ⇔ by default the browsers display text in black on a gray background
  - ⇔ change the background, text, and link colors

```
<BODY BGCOLOR="#000000" TEXT="#FFFFFF" LINK="9690CC">
```

RGB Color Model

– External Images, Sounds, and Animation

⇔ one may want to have an image open as a separate document when a user activates a link on either a word or smaller, inline version of the image included in a document (external image)

⇒ useful if you don't wish to slow down the loading of the main document with large inline image

```
<A HREF="MyImage.gif"> link anchor </A>
<A HREF="LargerImage.gif"><IMG SRC="SmallImage.gif"> </A>
<A HREF="영웅본색.mov"> 영웅 본색 (주윤발 주연) </A>
```

File Type	Extensions
plain text	.txt
HTML document	.html
GIF image	.gif
TIFF image	.tiff
X Bitmap image	.xbm
JPEG image	.jpg or .jpeg
PostScript file	.ps
AIFF sound file	.aiff
AU sound file	.au
WAV sound file	.wav
QuickTime movie	.mov
MPEG movie	.mpg or .mpeg

3. HTTP 프로토콜

- HTTP (HyperText Transfer Protocol) 란 ?
  - WWW에서의 하이퍼미디어 문서 (음성, 화상 등)의 전송을 위하여 쓰이는 프로토콜
  - MIME에 의하여 정의되는 모든 문서 양식 전송 가능
  - TCP를 기반으로 작성된 응용 프로토콜
    ⇔ GET, DELETE, POST, HEAD 등과 같은 응용 프로토콜 기능이 있다.
  - Well-Known Port Number : 80
  - 요구/응답 (request/response) 방식을 이용하여 동작
    ⇔ 원하는 프로토콜 기능(예: GET, DELETE, POST, HEAD 등)에 대하여 서비스 요구를 하면, 데이타 통신을 위한 TCP 연결이 만들어지고, 서버가 응답을 보내어 데이타 전송을 끝내면 자동적으로 연결이 끊어지게 된다.
- HTTP-NG
  - 각 문서 요구에 대하여 별도의 연결을 만들기 보다는 여러개의 문서 요구에 대하여 하나의 연결만을 만들도록 한다.
    ⇔ 하나의 TCP 연결 속네서 "session"이라 불리우는 여러 개의 가상적 연결을 만든다.
  - 데이타를 수신하는 동안에 기다릴 필요 없이 또 다른 문서 요청을 할 수 있다.

- **HTTP Request**
  - http://www.w3.org/pub/WWW/Protocols/
  - 명령어들
    - ⇔ GET *path* http/1.0 ₩r₩n₩r₩n (CRLFCRLF)
      - ⇒ *path*에 해당되는 HTML을 요청
    - ⇔ HEAD : retrieve the header but not the data
    - ⇔ POST
      - ⇒ pass data up to the server
      - ⇒ typically the URL refers to a script or program which will take the data
    - ⇔ S-HTTP (Secure HTTP) : adds encryption for security
- **HTTP Response**
  - version code reason
    - ⇔ HTTP/1.0 200 OK
  - Content-type: *MIME-type*
  - Refresh: seconds
- **Status Codes**
  - 200 = OK
  - 3xx = Minor client error - document in another location
  - 400 = Real client error - bad request, document not found, not allowed
  - 500 = Server error - service not implemented, service not available

- **MIME Type**
  - *content-type/sub-type [;aux-info]*

```
text/html
text/plain
text/plain ; charset=us-ascii
multipart/mixed; boundary=SpecialBoundaryString
application/acrobat
image/gif
video/mpeg
```

    "X-.." = experimental

  - since MIME type is in HTTP response header, the server is taking responsibility for identifying the type of data

- **HTTP 사용 예제**
  - **HTML text**

```
[pljuno:jhnang 1 ] telnet plsun 80
Trying 163.239.130.90 ...
Connected to plsun.sogang.ac.kr.
Escape character is '^]'.
GET / http/1.0

HTTP/1.0 200 Document follows
Date: Thu, 26 Sep 1996 12:55:54 GMT
Server: NCSA/1.4.2
Content-type: text/html
Last-modified: Tue, 20 Aug 1996 05:39:28 GMT
Content-length: 1810

<html>
<head>
<title>Parallel Programming Languages and Systems </title>
</head>

<body bgcolor="#FFFFFF">
<center>
<img src="pics/title1.gif">
<img width=520 src="gif/rainban.gif">
<h1>
.....
```

– Dual Error Reporting

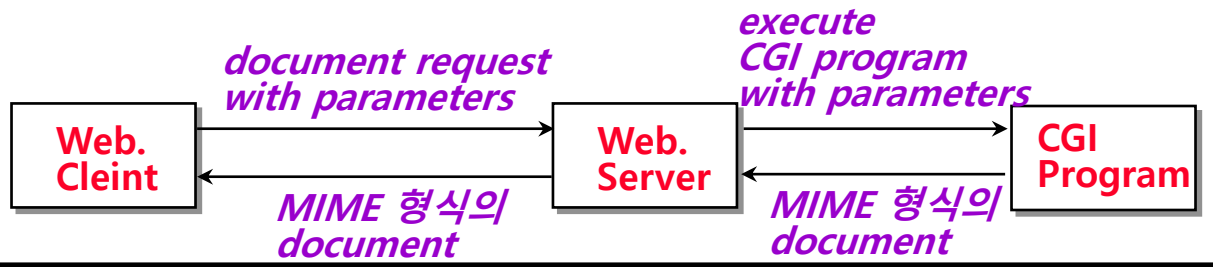
```
[pljuno:jhnang 1 ] telnet plsun 80
Trying 163.239.130.90 ...
Connected to plsun.sogang.ac.kr.
Escape character is '^]'.
GET /~jhnang/lalala http/1.0

HTTP/1.0 404 Not Found
Date: Thu, 26 Sep 1996 12:58:46 GMT
Server: NCSA/1.4.2
Content-type: text/html

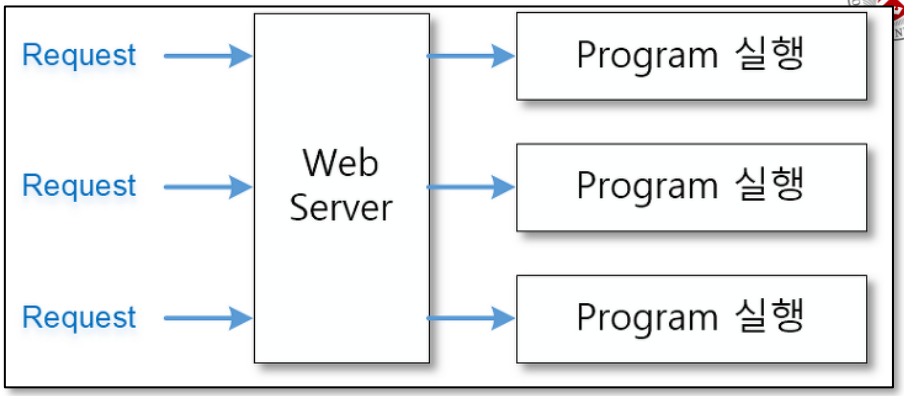
<HEAD><TITLE>404 Not Found</TITLE></HEAD>
<BODY><H1>404 Not Found</H1>
The requested URL /~jhnang/lalala was not found on this server.<P>
</BODY>
Connection closed by foreign host.
[pljuno:jhnang 2 ] _
```

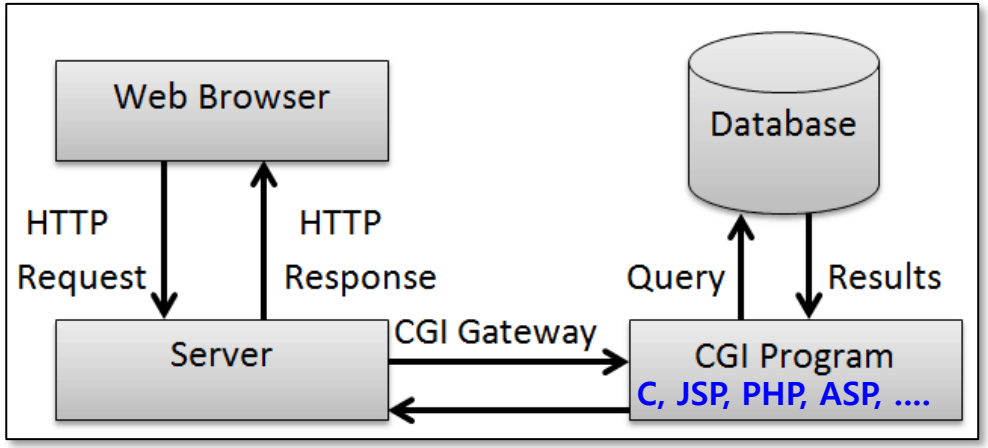
4. CGI 프로그래밍

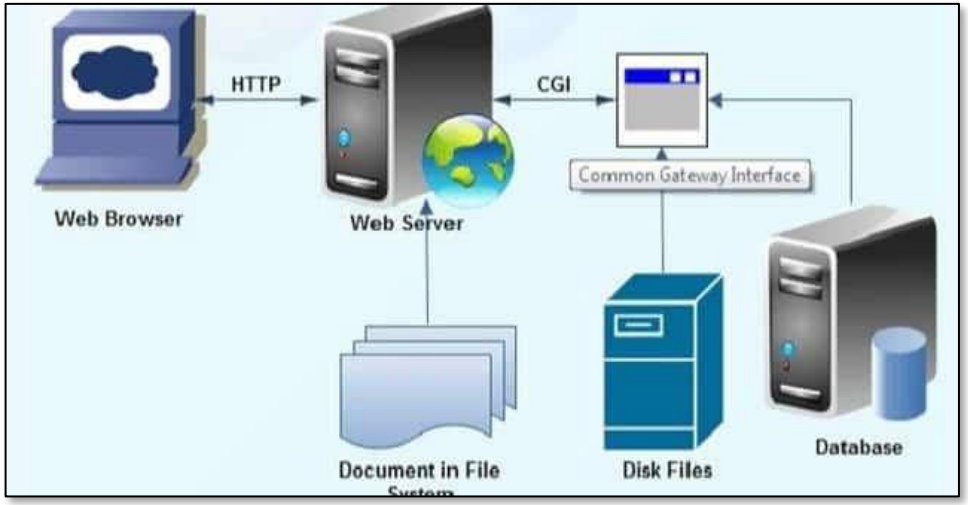
- CGI (Common Gateway Interface)란 ?
  - Web. 하이퍼미디어 시스템과 외부 응용 프로그램이 서로 연계되기 위한 인코딩, 디코딩 방법과 데이타 교환 방법을 정의한 표준
    - ⇔ Common : 플랫폼에 관계 없이 각 클라이언트와 서버, 그리고 게이트웨이 프로그램이 공유하는 표준
    - ⇔ Gateway : 이질적인 (heterogeneous) 두 시스템의 통로 역할을 맡고 있음
    - ⇔ Interface : 개발자가 HTTP 서버의 전체적인 프로그램 구조를 모르더라도 입출력에 관련된 부분만 이해하면 게이트웨이 프로그램을 개발 가능
  - Virtual Document, not Text file
- CGI 프로그램란 ?
  - 독립적으로 실행될 수 있는 실행 프로그램
  - C/C++, ASP (Active Server Page, microsoft), JSP (Java Server Page), PHP 등으로 작성 가능
- Why CGI ?
  - 사용자 질의에 따른 동적인 문서 (HTML) 생성 가능 ➔ 외부 프로그램에 의한 Web 기능 확장
    - ⇔예) 정보 검색에 대한 질의 화면 및 결과 화면 (HTML 문서)을 생성

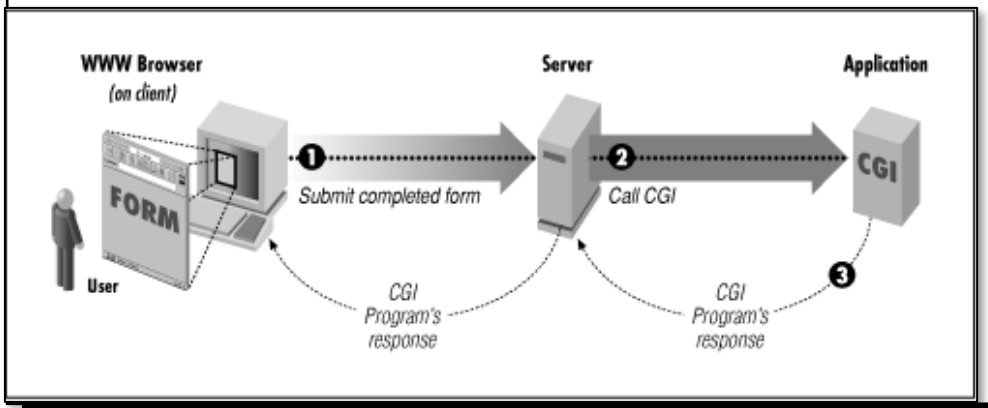


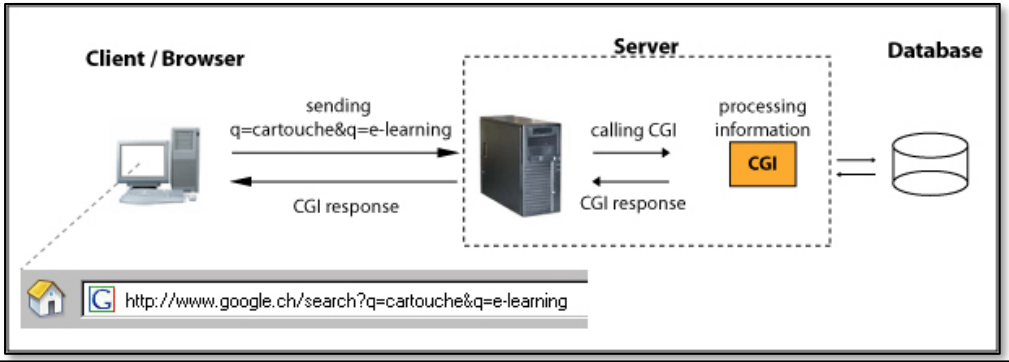
- **CGI (Common Gateway Interface)**
  - ✓ 외부 프로그램의 필요 이유
    - ➔ Web 서비스의 확장성
  - ✓ 사용자 질의에 대한 결과를 보여주는 동적 웹 페이지 (dynamic web page) 생성을 위한 브라우저 <-> 웹 서버 <-> 외부 프로그램 사이의 파라메터 전달 방법 (HTTP 프로토콜 이용)



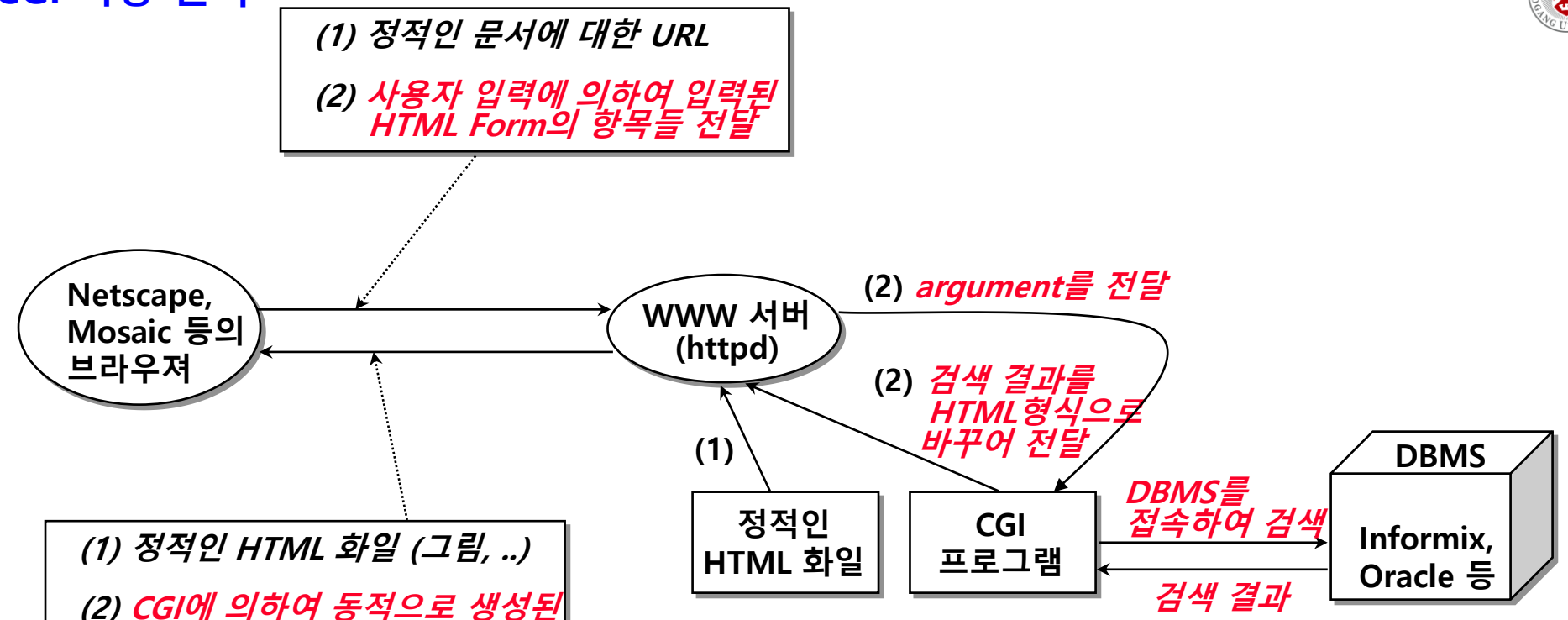


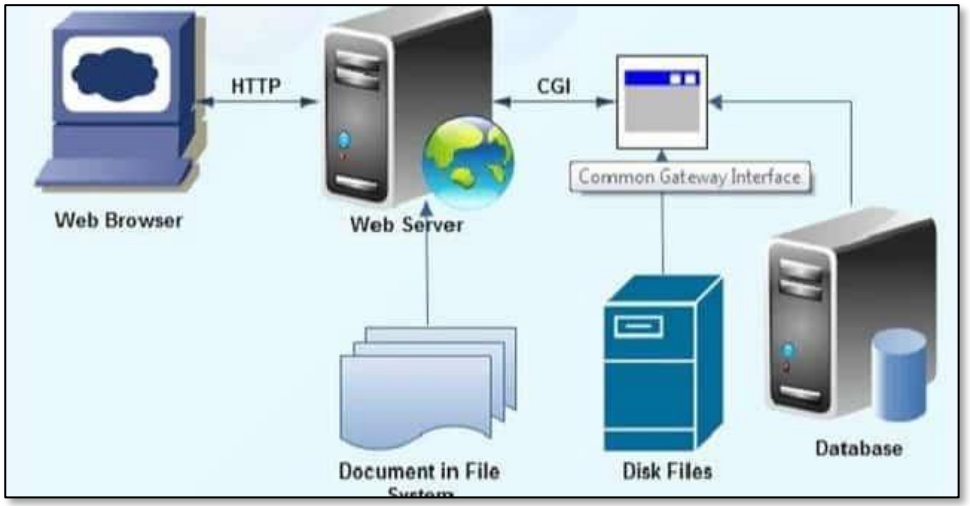






• CGI 작동 원리

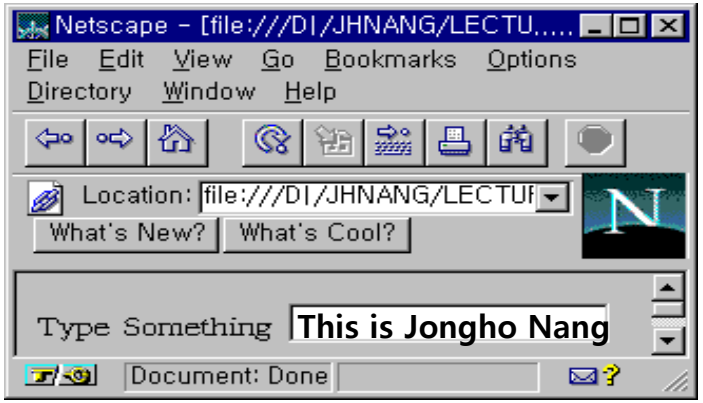

(1) 정적인 문서에 대한 URL
(2) 사용자 입력에 의하여 입력된 HTML Form의 항목들 전달
Netscape, Mosaic 등의 브라우저
WWW 서버 (httpd)
(2) argument를 전달
(2) 검색 결과를 HTML형식으로 바꾸어 전달
(1)
정적인 HTML 화일
CGI 프로그램
DBMS를 접속하여 검색
검색 결과
DBMS
Informix, Oracle 등
(1) 정적인 HTML 화일 (그림, ..)
(2) CGI에 의하여 동적으로 생성된 HTML 화일


HTTP
CGI
Common Gateway Interface
Web Browser
Web Server
Document in File System
Disk Files
Database

• CGI 작동 예

http://cspro.sogang.ac.kr/~jhnang/echo-test.html

```
<BODY>
<FORM action="http://cspro.sogang.ac.kr/~jhnang/cgi-bin/echo-test.cgi" method=GET>
Type Something <INPUT TYPE=text Name="input_lala">
</FORM>
</BODY>
```



http://plsun.sogang.ac.kr/cgi-bin/echo-test?input_lala=This+is+Jongho+Nang&

WWW 서버 (httpd)

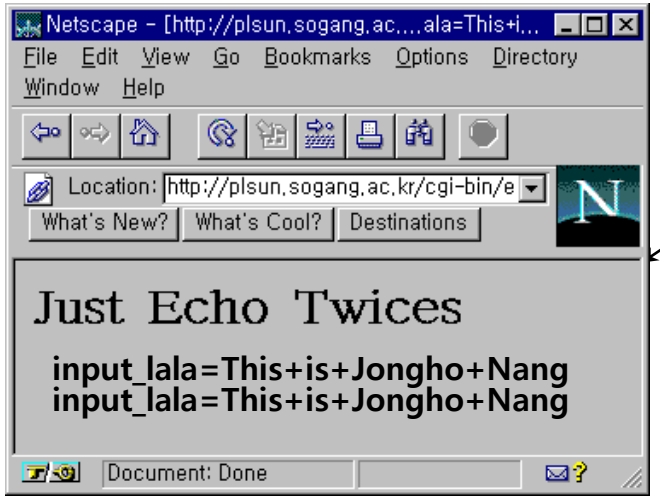
fork()

$QUERY_STRING= "input_lala= This+is+Jongho+Nang"

```
Content-type: text/html

<HTML><BODY>
<H1>Just Echo Twices </H1>
input_lala=This+is+Jongho+Nang
input_lala=This+is+Jongho+Nang
</BODY> </HTML>
```

"echo-test.cgi" cgi 프로그램에 의하여 생성된 HTML 화일



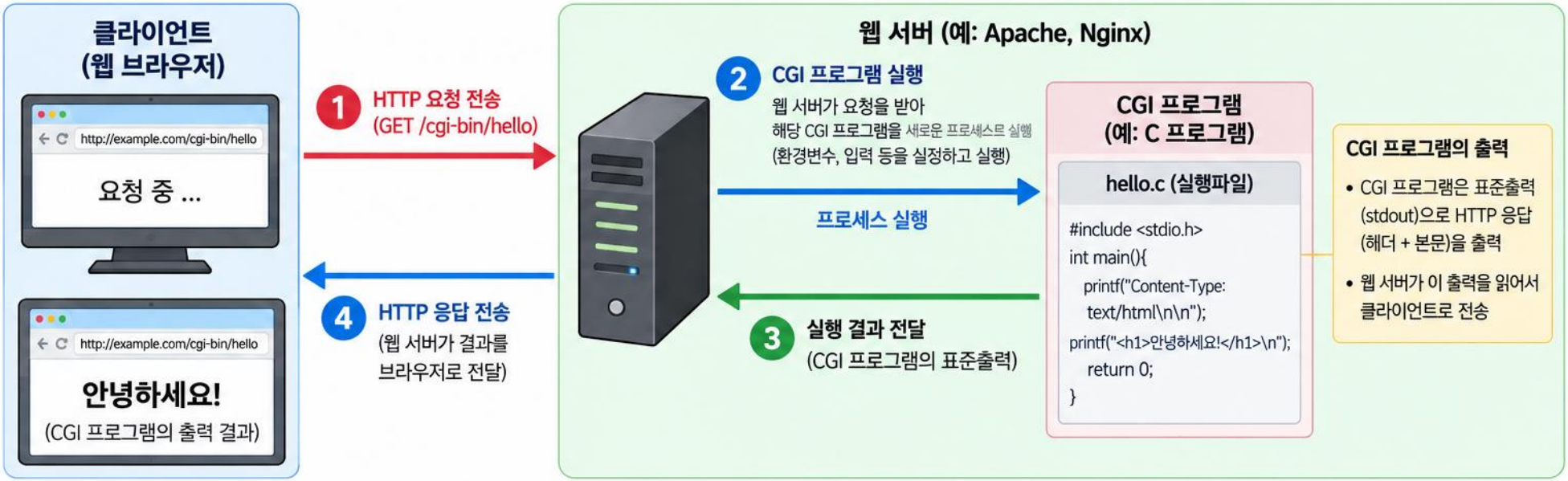

```
#!/bin/sh
echo Content-type: text/html
echo
echo "<HTML><BODY>"
echo "<H1>Just Echo Twices </H1>"
echo "$QUERY_STRING"
echo "$QUERY_STRING"
echo "</BODY></HTML>"
```

"~jhnang/cgi-bin/echo-test.cgi"

일반적인 CGI 프로그램 (예: C로 작성한 CGI)의 동작 방식

CGI 프로그램도 PHP와 마찬가지로, 웹 서버가 프로그램을 실행하고 그 출력 결과를 받아서 브라우저로 전송합니다.

CGI 프로그램은 브라우저와 직접 통신하지 않고, 웹 서버를 통해 실행되고, 그 표준출력을 웹 서버가 받아 HTTP 응답으로 브라우저에 전달합니다.



CGI 프로그램이 출력하는 내용 (예: C 프로그램)

```c
#include <stdio.h>
int main(){
  printf("Content-Type: text/html\n\n");   // HTTP 헤더
  printf("<h1>안녕하세요!</h1>\n");         // HTTP 본문 (HTML)
  return 0;
}
```

- CGI 프로그램은 표준출력으로 HTTP 응답 형식(헤더 + 본문)을 출력합니다.
- 웹 서버(Apache, Nginx 등)가 이 출력을 읽어서 클라이언트(브라우저)로 전송합니다.
- CGI 프로그램은 브라우저와 직접 소켓 통신을 하지 않습니다.
- 브라우저는 웹 서버와만 HTTP 통신을 합니다.
- (참고) PHP의 CGI 모드 (php-cgi), 다른 언어로 만든 CGI 프로그램(C, Perl, Python 등) 모두 동일한 방식입니다.

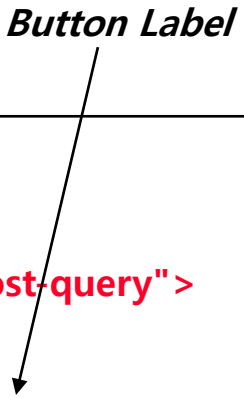
결론: 일반적인 CGI 프로그램 (예: C로 작성한 CGI 프로그램)도 웹 서버를 통해 실행되고, 그 출력 결과는 웹 서버가 받아서 HTTP 응답으로 브라우저에 전송합니다. CGI 프로그램이 브라우저와 직접 통신하지는 않습니다.

4.1 Fill-Out Form Tag

- Fill-Out From Tag (클라이언트에서 사용자의 입력을 받는 인터페이스)
  - <form> .... </form> : HTML 문서내에 form문을 지정한다. 아래의 tag들은 모두 이 tag안에 오게 된다.
    - ⇔ action : CGI 프로그램의 URL
    - ⇔ method : GET 혹은 POST
    - ⇔ enctype : 인코드 형식으로서 POST 경우만 가능하다.
      - ⇒ accplication/x-www-form-urlemcoded : 일반적인 경우
      - ⇒ multipart/form-data : 화일 upload인 경우
  - <input> : 입력 filed를 지정한다.
    - ⇔ type : text, password, checkbox, radio, submit, reset 중의 하나
    - ⇔ name : 이름을 지정한다.
    - ⇔ value : filed의 값을 지정한다.
    - ⇔ checked : checkbox와 radio에서 사용한다.
    - ⇔ size : filed의 크기를 지정한다.
    - ⇔ maxlength : 실제 입력할 수 있는 최대 문자수를 지정한다.
  - <select> .... </select> : 여러 항목 리스트 중에서 선택을 지정한다.
    - ⇔ name : 이름을 지정한다.
    - ⇔ size : 항목 수를 지정한다.
    - ⇔ multiple : 다중 선택을 가능하게 한다.

- <option> : <select> tag안에 오며, 각 항목을 나타낸다.
  ⇔ selected : 기존적으로 선택되게 한다.
- <textarea> .... </textarea> : 다수의 라인을 갖는 텍스트 입력 filed를 지정한다.
  ⇔ name : 이름을 지정한다.
  ⇔ rows : 행의 크기를 지정한다.
  ⇔ cols : 열의 크기를 지정한다.

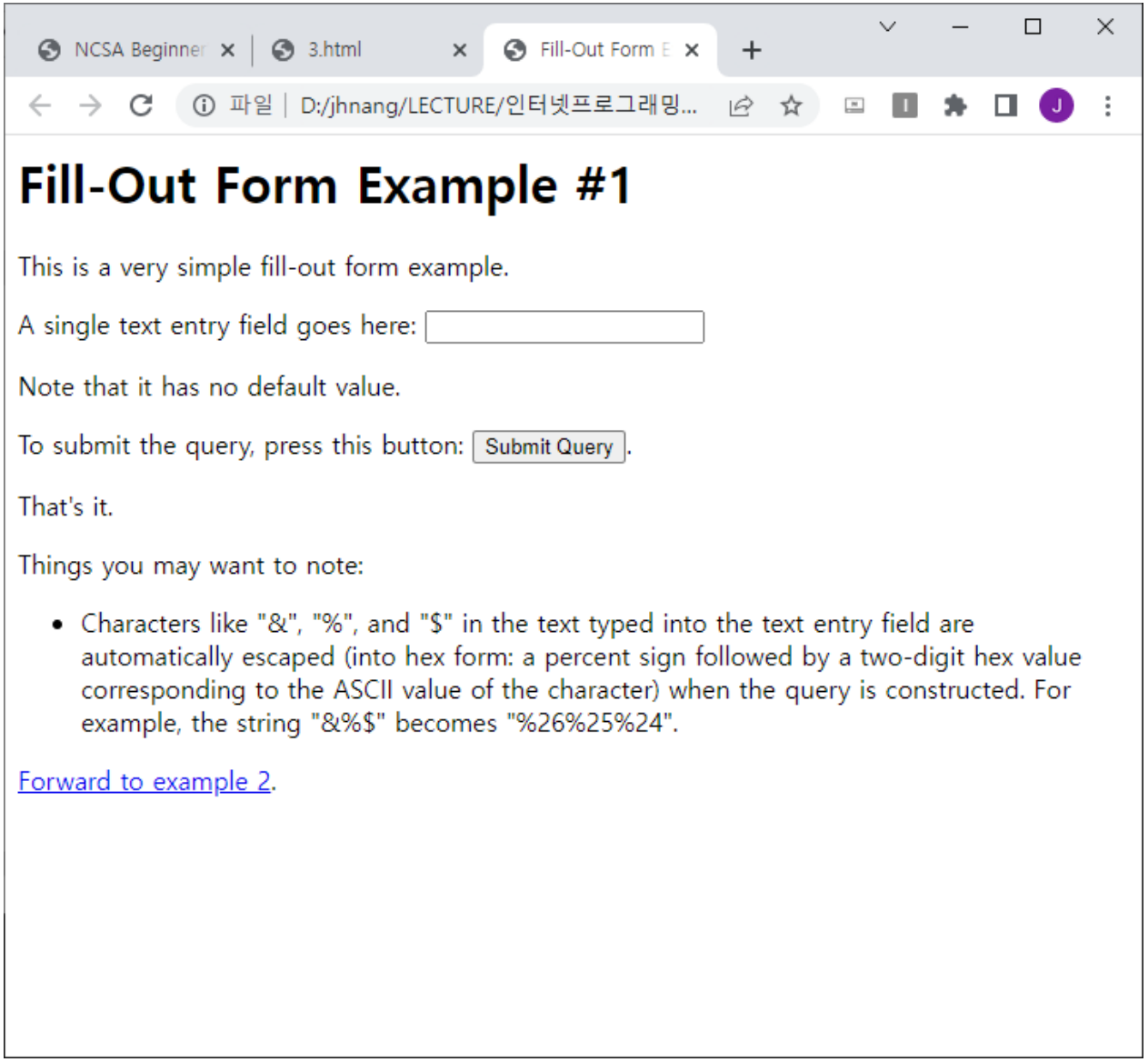
- 예제 1 : HTML Source




```
<TITLE>Fill-Out Form Example #1</TITLE>
<H1>Fill-Out Form Example #1</H1>
This is a very simple fill-out form example. <P>
<FORM  METHOD="POST" ACTION="http://hoohoo.ncsa.uiuc.edu/cgi-bin/post-query">
A single text entry field goes here: <INPUT NAME="entry"> <P>
Note that it has no default value. <P>
To submit the query, press this button: <INPUT TYPE="submit" VALUE="Submit Query">. <P>
</FORM>

That's it. <P>
Things you may want to note: <P>
<UL>
<LI> Characters like "&amp;amp;", "%", and "$" in the text typed into the text entry field are
     automatically escaped (into hex form: a percent sign followed by a two-digit hex value
     corresponding to the ASCII value of the character) when the query is constructed.
     For example, the string "&amp;amp;%$" becomes "%26%25%24".
</UL>
<A HREF="example-2.html">Forward to example 2</A>. <P>
```

– 예제 1 : Browser 출력



– 예제 2 : HTML Source

http://cspro.sogang.ac.kr/~jhnang/pizza_order2020.html

```
<TITLE>Fill-Out Form Example #7</TITLE>
<H1>Fill-Out Form Example #7</H1>
This is another fill-out form example, with toggle buttons. <P>
<HR>
<FORM METHOD="GET" ACTION="http://cspro.sogang.ac.kr/~jhnang/cgi-bin/pizza_order2020Echo.php">
<H2>Godzilla's Pizza -- Internet Delivery Service, Part II</H2>
Type in your street address: <INPUT NAME="address"> <P>
Type in your phone number: <INPUT NAME="phone"> <P>
Which toppings would you like? <P>
<OL>
<LI> <INPUT TYPE="checkbox" NAME="topping" VALUE="pepperoni"> Pepperoni.
<LI> <INPUT TYPE="checkbox" NAME="topping" VALUE="sausage"> Sausage.
<LI> <INPUT TYPE="checkbox" NAME="topping" VALUE="anchovies"> Anchovies.
</OL>
How would you like to pay?  Choose any one of the following: <P>
<OL>
<LI> <INPUT TYPE="radio" NAME="paymethod" VALUE="cash" CHECKED> Cash.
<LI> <INPUT TYPE="radio" NAME="paymethod" VALUE="check"> Check.
<LI> <I>Credit card:</I>
<UL>
<LI> <INPUT TYPE="radio" NAME="paymethod" VALUE="mastercard"> Mastercard.
<LI> <INPUT TYPE="radio" NAME="paymethod" VALUE="visa"> Visa.
<LI> <INPUT TYPE="radio" NAME="paymethod" VALUE="americanexpress"> American Express.
</UL>
</OL>
```

http://cspro.sogang.ac.kr/~jhnang/cgi-bin/pizza_order2026Echo.php

```
<html>
<body>

<?php
echo "<h2> NANG Your Input:</h2>";
echo "Name : "  ;
echo $_GET["name"];
echo "<br>";
echo "Address : "  ;
echo $_GET["address"] ;
echo "<br>";
echo "Phone Number : "  ;
echo $_GET["phone"];
echo "<br>";
echo "Topping : " ;
echo $_GET["topping"] ;
echo "<br>";
echo "Pay Method : "  ;
echo $_GET["paymethod"];
?>

</body>
</html>
```

```
<html>
<body>

<?php
echo "<h2> NANG Your Input:</h2>";
echo "Name : "  ;
echo $_GET["name"];
echo "<br>";
echo "Address : "  ;
echo $_GET["address"] ;
echo "<br>";
echo "Phone Number : "  ;
echo $_GET["phone"];
echo "<br>";
echo "Topping : " ;
echo $_GET["topping"] ;
echo "<br>";
echo "Pay Method : "  ;
echo $_GET["paymethod"];
?>

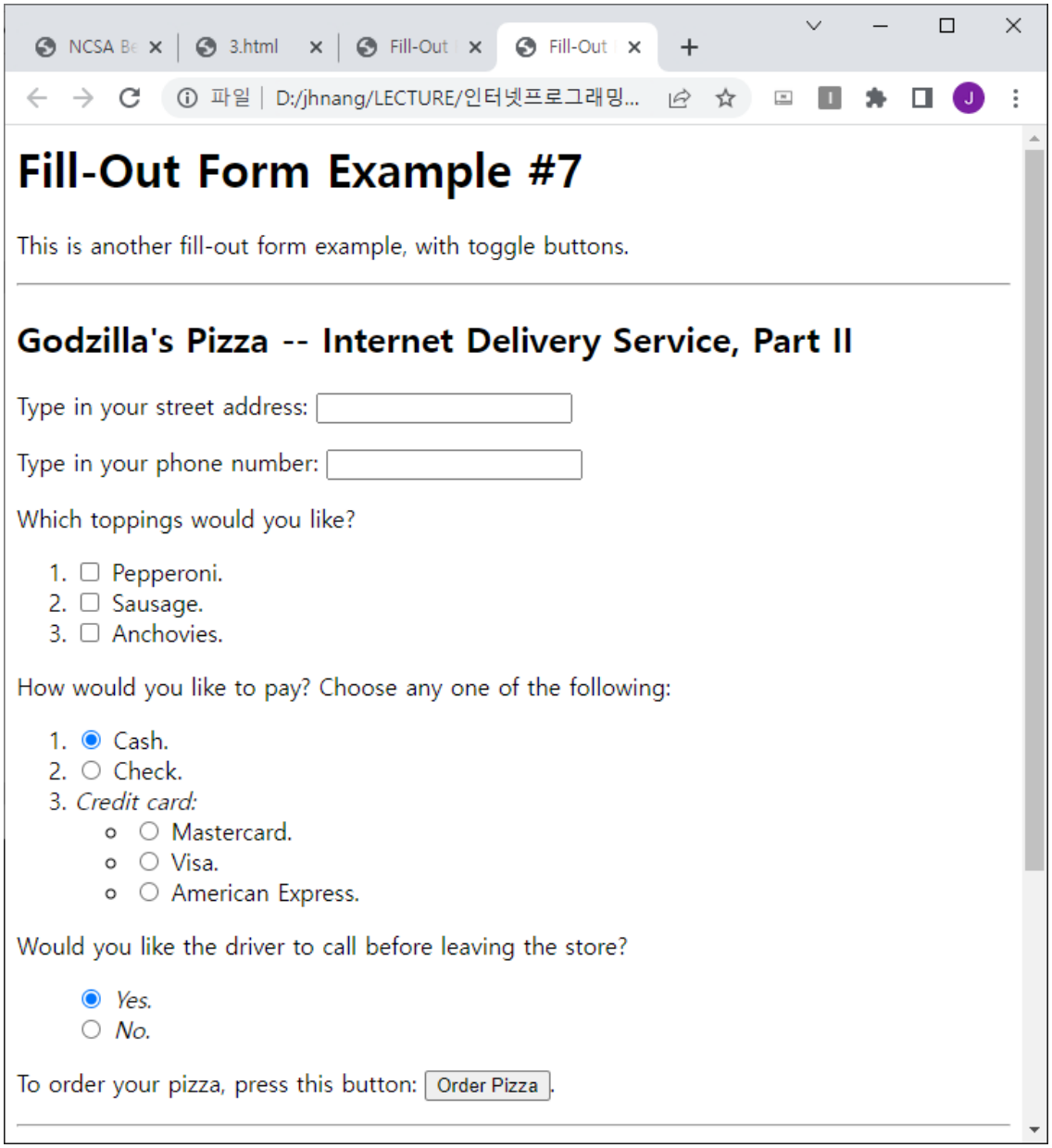
</body>
</html>
```

```
Would you like the driver to call before leaving the store? <P>
<DL>
<DD> <INPUT TYPE="radio" NAME="callfirst" VALUE="yes" CHECKED> <I>Yes.</I>
<DD> <INPUT TYPE="radio" NAME="callfirst" VALUE="no"> <I>No.</I>
</DL>
To order your pizza, press this button: <INPUT TYPE="submit"VALUE="Order Pizza">. <P>

</FORM>
<HR> <P>
Things you may want to note: <P>
<UL>
<LI> Radio buttons (type <CODE>RADIO</CODE>) are one-of-many input
    elements.  The "many" is all radio buttons in the form with the
    same <CODE>NAME</CODE>.
<LI> In other ways, <CODE>RADIO</CODE> is the same as
    <CODE>CHECKBOX</CODE>.
<LI> Multiple sets of radio buttons (radio buttons with the same
    <CODE>NAME</CODE>) can be in a single form.  This may require
    careful document/interface design to keep things obvious -- don't
    intermix radio buttons of different <CODE>NAME</CODE> values, or
    the user won't understand how the interface works.
</UL>

<A HREF="example-6.html">Back to example 6</A> or <A
HREF="example-8.html">forward to example 8</A>. <P>
```

– 예제 2 : Browser 출력



– 예제 3 : HTML Source

http://cspro.sogang.ac.kr/~jhnang/Example10.html

```
<TITLE>Fill-Out Form Example #10</TITLE>
<H1>Fill-Out Form Example #10</H1>
This is another fill-out form example, with scrolled selection lists
with both one and <CITE>n</CITE> of many behavior. <P>
<HR>
<FORM METHOD="POST" ACTION="http://cspro.sogang.ac.kr/~jhnang/cgi-bin/post_query.cgi">">
<H2>Your Daily Planner</H2>
What would you like to do today?
<SELECT NAME="what-to-do" SIZE=10>
<OPTION VALUE="drink">Drink Coffee
<OPTION VALUE="read" SELECTED>Read A Book
<OPTION value=walk>Take A Walk
<OPTION> Chop Down Rainforests
<OPTION> Buy A Bagel
<OPTION>Watch TV
<OPTION>                Sell Art
<OPTION>Buy Jewelry
<OPTION>Attend A Concert
<OPTION>Read Poetry
<OPTION>    Exercise
<OPTION>Whittle
<OPTION>Cook Stir Fry
<OPTION>Sip Espresso
<OPTION value="Pig Out">Munch Teddy Grahams
</SELECT> <P>
```

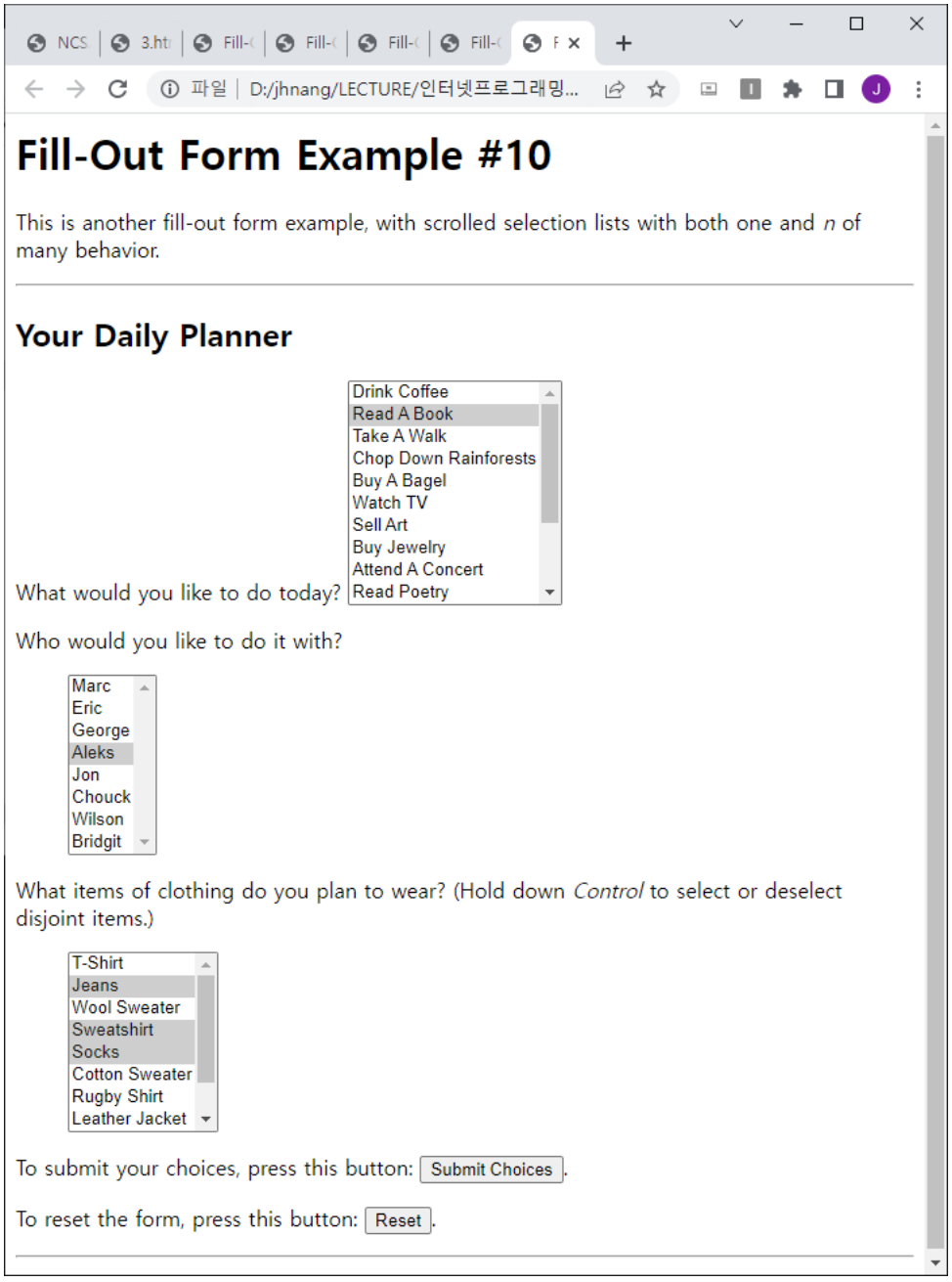
```
% gcc -o post_query.cgi post_query.c
```

http://cspro.sogang.ac.kr/~jhnang/cgi-bin/post_query.c

– 예제 3 : HTML Source (계속)

```
Who would you like to do it with? <P>
<DL>
<DD>
<SELECT NAME="who-to-do-it-with" SIZE=8>
<OPTION>Marc      <OPTION>Eric      <OPTION>George    <OPTION SELECTED>Aleks
<OPTION>Jon       <OPTION>Chouck    <OPTION> Wilson   <OPTION> Bridgit
</SELECT>
</DL>
What items of clothing do you plan to wear?  (Hold down <I>Control</I>
to select or deselect disjoint items.) <P>
<DL>
<DD>
<SELECT NAME="what-to-wear" MULTIPLE SIZE=8>
<OPTION>T-Shirt    <OPTION SELECTED>Jeans      <OPTION>Wool Sweater
<OPTION SELECTED>Sweatshirt          <OPTION SELECTED>Socks
<OPTION>Cotton Sweater                          <OPTION>Rugby Shirt
<OPTION>Leather Jacket       <OPTION>Boots      <OPTION>Running Shoes
</SELECT>
</DL>
To submit your choices, press this button: <INPUT TYPE="submit" VALUE="Submit Choices">. <P>
To reset the form, press this button: <INPUT TYPE="reset" VALUE="Reset">.
</FORM>
<HR> <P>
Things you may want to note: <P>
<UL>
<LI> Blah.
</UL>
```

– 예제 3 : Browser 출력



4.2 CGI Server-Side Programming

https://httpd.apache.org/docs/2.4/ko/howto/cgi.html

- CGI 인수 Encoding Rule
  - form에서 입력하는 사항은 encoding되어 CGI 프로그램에게 전달된다.
  - Rule
    - ⇔ Name/Value Pair : &, =
      - ⇒ Name1=*Value1*&Name2=*Value2*& ....
    - ⇔ Space : '+'
    - ⇔ Special Characters : '%'Hex ASCII Code
  - 예) name1=First, name2 = Is it Second? That's right!
    - ⇔ name1=First&name2=Is+it+Second%3F+That%26s+right%21
- CGI로의 인수 전달
  - GET
    - ⇔ packaging the name=values into the URL and then passing it to the CGI script as the QUERY_STRING environment variable
    - ⇔ CGI 프로그램은 환경 변수 QUERY_STRING을 통하여 암호화된 form 입력을 받는다.
      - ⇒ <FORM METHOD=GET ACTION="http://www.cc.ukans.edu/cgi-bin/post-query">
      - ⇒ GET /cgi-bin/post-query?name1=First&name2=Is+it+Second%3F+That%26s+right%21 HTTP/1.0
  - POST
    - ⇔ CGI프로그램은 stdin상에서 암호화된 form 입력을 받는다.
    - ⇔ 환경 변수 CONTENT_LENGTH를 이용하여 얼마나 많은 데이타를 stdin을 통하여 받을 것인지 알 수 있음

- **CGI의 출력**
  - ⇔ CGI는 세가지의 헤더 - Location, Content-type, Status-를 정의 하고 있다.
  - ⇔ 서버는 CGI의 출력인 이 헤더 정보를 HTTP 응답 헤더 (HTTP response header)에 적절히 삽입하여 클라이언트에게 넘겨준다.
  - **Location**
    - ⇔ 자료를 넘겨주는 대신에 자료가 있는 위치를 넘겨준다.
    - ⇔ 예) Location : http://host.domain/index.html
  - **Content-type**
    - ⇔ 넘겨줄 자료의 형식을 지정하고 자료를 넘겨 준다.
    - ⇔ 자료 형식은 MIME (Multipurpose Internet Mail Extensions) 형식을 따른다.
    - ⇔ 예) Content-type: text/html
      <html> ..... </html>
  - **Status**
    - ⇔ 결과 코드를 지정한다.
    - ⇔ 예) Status: 500 Internal Server Error
  - **NPH (No-Parse Header)**
    - ⇔ HTTP 응답 헤더 전체를 CGI 프로그램이 출력시켜주는 형태이다.
    - ⇔ 예)

```
HTTP/1.0 200 OK
Content-type: text/html
<html> .... </html>
```

- **CGI 환경 변수 :**
  - **GET**

http://cspro.sogang.ac.kr/~jhnang/test-envGet.html

```
<BODY>
<FORM action="http://cspro.sogang.ac.kr/~jhnang/cgi-bin/test-env.cgi" method=GET>
Type Something <INPUT TYPE=text Name="input_lala">
</FORM>
</BODY>
```

Web. Browser 출력

```
DOCUMENT_ROOT=/usr/local/etc/httpd/htdocs
GATEWAY_INTERFACE=CGI/1.1
HTTP_ACCEPT=image/gif, image/x-xbitmap, image/jpeg, image/pjpeg, */*
HTTP_CONNECTION=Keep-Alive
HTTP_HOST=plsun.sogang.ac.kr
HTTP_REFERER=file:///D|/JHNANG/LECTURE/Internet/HTML/CGI 예제.html
HTTP_USER_AGENT=Mozilla/3.0 (Win95; I)
PATH=/bin:/usr/bin:/usr/etc:/usr/ucb
QUERY_STRING=input_lala=This+is+Sogang
REMOTE_ADDR=163.239.135.93
REMOTE_HOST=plmercury
REQUEST_METHOD=GET
SCRIPT_NAME=/cgi-bin/test-env
SERVER_NAME=plsun.sogang.ac.kr
SERVER_PORT=80
SERVER_PROTOCOL=HTTP/1.0
SERVER_SOFTWARE=NCSA/1.4.2
```

http://cspro.sogang.ac.kr/~jhnang/cgi-bin/test-env.cgi

```
#!/bin/sh
#
# Prints out all of the environment variables as used
#
echo Content-type: text/plain
echo
env
```

– POST

http://cspro.sogang.ac.kr/~jhnang/test-envPost.html

```
<BODY>
<FORM action="http://cspro.sogang.ac.kr/~jhnang/cgi-bin/test-env.cgi " method="POST">
아무 글자나 치세요 <INPUT TYPE=text Name="input_lala">
</FORM>
</BODY>
```

Web. Browser 출력

```
CONTENT_LENGTH=21
CONTENT_TYPE=application/x-www-form-urlencoded
DOCUMENT_ROOT=/usr/local/etc/httpd/htdocs
GATEWAY_INTERFACE=CGI/1.1
HTTP_ACCEPT=image/gif, image/x-xbitmap, image/jpeg, image/pjpeg, */*
HTTP_CONNECTION=Keep-Alive
HTTP_HOST=plsun.sogang.ac.kr
HTTP_REFERER=file:///D|/JHNANG/LECTURE/Internet/HTML/CGI 예제.html
HTTP_USER_AGENT=Mozilla/3.0 (Win95; I)
PATH=/bin:/usr/bin:/usr/etc:/usr/ucb
QUERY_STRING=
REMOTE_ADDR=163.239.135.93
REMOTE_HOST=plmercury
REQUEST_METHOD=POST
SCRIPT_NAME=/cgi-bin/test-env
SERVER_NAME=plsun.sogang.ac.kr
SERVER_PORT=80
SERVER_PROTOCOL=HTTP/1.0
SERVER_SOFTWARE=NCSA/1.4.2
```

http://cspro.sogang.ac.kr/~jhnang/cgi-bin/test-env.cgi

```
#!/bin/sh
#
# Prints out all of the environment variables as used
#
echo Content-type: text/plain
echo
env
```

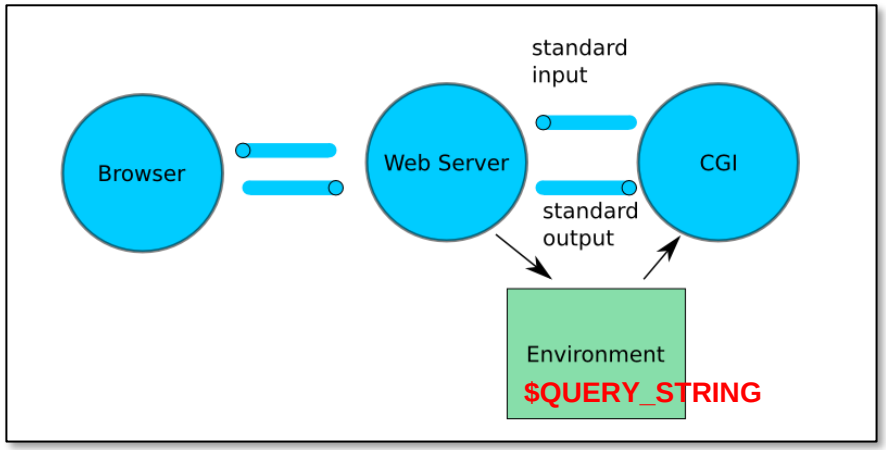
– C 언어를 이용한 CGI 프로그램

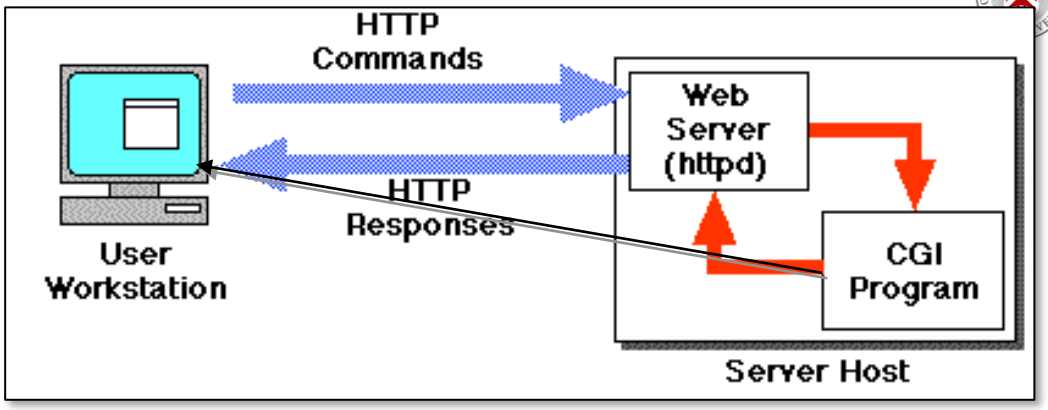
```
#include <stdio.h>
#define MAX_ENTRIES 10000
typedef struct {
    char *name;
    char *val;
} entry;
char *getenv();
char *makeword(char *line, char stop);
char *fmakeword(FILE *f, char stop, int *len);
char x2c(char *what);
void unescape_url(char *url);
void plustospace(char *str);

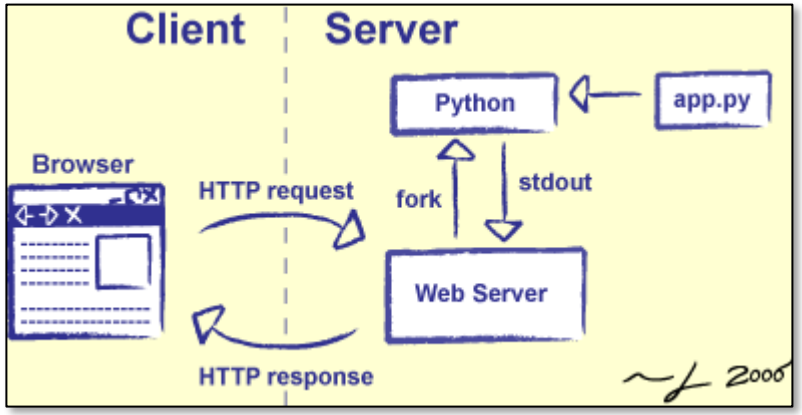
main(int argc, char *argv[]) {
    entry entries[MAX_ENTRIES];
    register int x,m=0;
    int cl;
    printf("Content-type: text/html%c%c",10,10);
    if(strcmp(getenv("REQUEST_METHOD"),"POST")) {
        printf("This script should be referenced with a METHOD of POST.₩n");
        exit(1);
    }
    if(strcmp(getenv("CONTENT_TYPE"),"application/x-www-form-urlencoded")) {
        printf("This script can only be used to decode form results. ₩n");
        exit(1);
    }
```

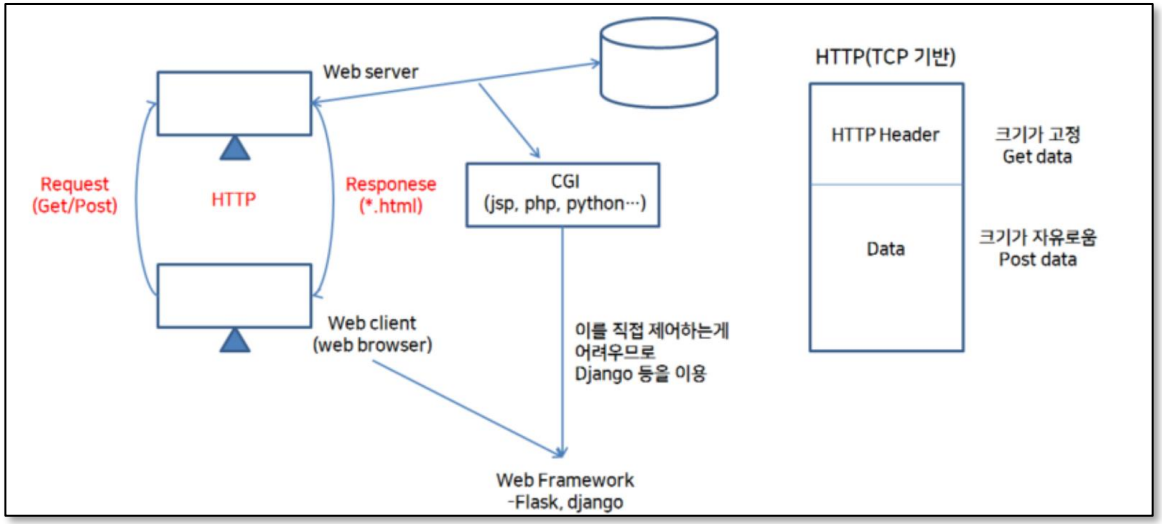
```
    cl = atoi(getenv("CONTENT_LENGTH"));
    for(x=0;cl && (!feof(stdin));x++) {
            m=x;
            entries[x].val = fmakeword(stdin,'&',&cl);
            plustospace(entries[x].val);
            unescape_url(entries[x].val);
            entries[x].name = makeword(entries[x].val,'=');
    }
    printf("<H1>Query Results</H1>");
    printf("You submitted the following ₩
                name/value pairs:<p>%c",10);
    printf("<ul>%c",10);
    for(x=0; x <= m; x++)
    printf("<li> <code>%s = %s</code>%c",
                entries[x].name, entries[x].val,10);
        printf("</ul>%c",10);
    }
```

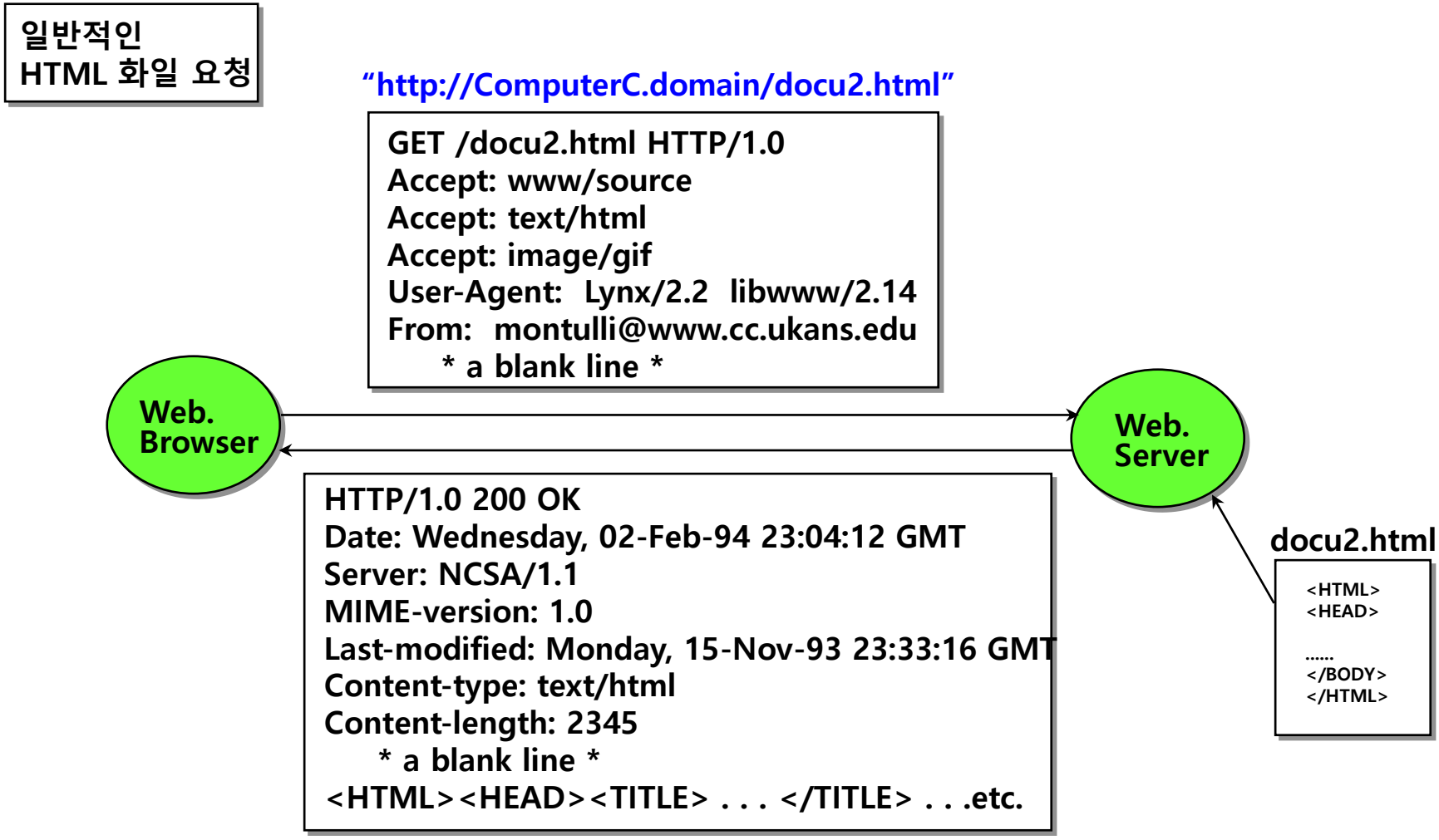
http://cspro.sogang.ac.kr/~jhnang/post_query.c


standard
input
Browser
Web Server
CGI
standard
output
Environment
$QUERY_STRING


HTTP
Commands
Web
Server
(httpd)
HTTP
Responses
User
Workstation
CGI
Program
Server Host


Client
Server
Python
app.py
Browser
HTTP request
fork
stdout
Web Server
HTTP response
2006


Web server
HTTP(TCP 기반)
HTTP Header
크기가 고정
Get data
Request
(Get/Post)
HTTP
Responese
(*.html)
CGI
(jsp, php, python…)
Data
크기가 자유로움
Post data
Web client
(web browser)
이를 직접 제어하는게
어려우므로
Django 등을 이용
Web Framework
-Flask, django


일반적인
HTML 화일 요청
"http://ComputerC.domain/docu2.html"
GET /docu2.html HTTP/1.0
Accept: www/source
Accept: text/html
Accept: image/gif
User-Agent: Lynx/2.2 libwww/2.14
From: montulli@www.cc.ukans.edu
* a blank line *
Web. Browser
Web. Server
HTTP/1.0 200 OK
Date: Wednesday, 02-Feb-94 23:04:12 GMT
Server: NCSA/1.1
MIME-version: 1.0
Last-modified: Monday, 15-Nov-93 23:33:16 GMT
Content-type: text/html
Content-length: 2345
* a blank line *
<HTML><HEAD><TITLE> . . . </TITLE> . . .etc.
docu2.html
<HTML>
<HEAD>
.....
</BODY>
</HTML>

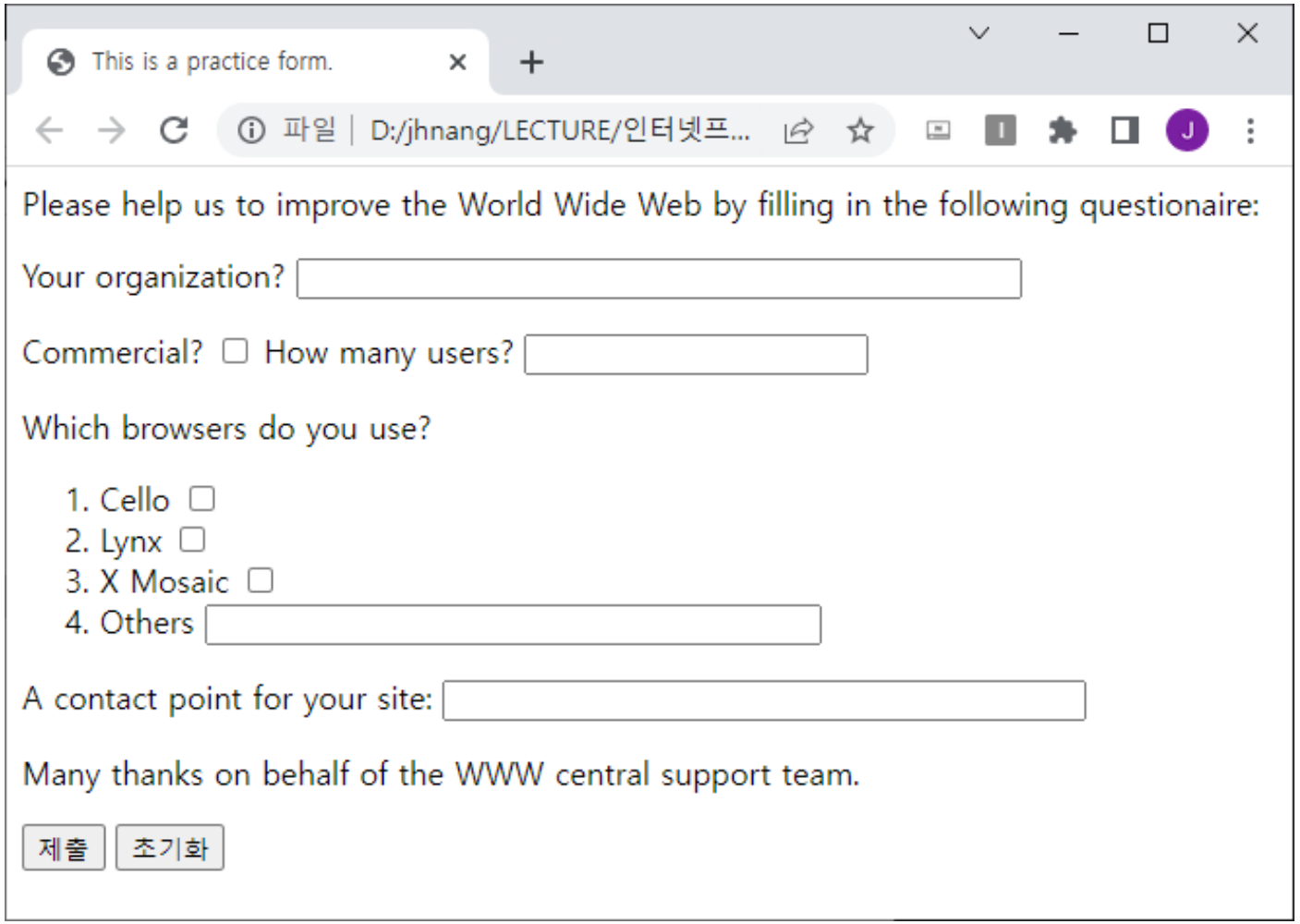
CGI 프로그램 수행 요청

HTML Source

http://cspro.sogang.ac.kr/~jhnang/CGI_RequestExample.html

```
<html>
<head>
<title>This is a practice form.</title>
<link rev=made href="mailto:grobe@kuhub.cc.ukans.edu">
</head>
<body>
<FORM METHOD=GET
      ACTION="http://cspro.sogang.ac.kr/~jhnang/cgi-bin/post_query.cgi">
Please help us to improve the World Wide Web by filling in the
following questionaire:
   <P>Your organization? <INPUT NAME="org" TYPE=text SIZE="48">
   <P>Commercial? <INPUT NAME="commerce" TYPE=checkbox>
         How many users? <INPUT NAME="users" TYPE=int>
   <P>Which browsers do you use?
   <OL>
    <LI>Cello <INPUT NAME="browsers" TYPE=checkbox VALUE="cello">
    <LI>Lynx <INPUT NAME="browsers" TYPE=checkbox VALUE="lynx">
    <LI>X Mosaic <INPUT NAME="browsers" TYPE=checkbox VALUE="mosaic">
    <LI>Others <INPUT NAME="others" SIZE=40>
   </OL>
A contact point for your site: <INPUT NAME="contact" SIZE="42">
<P>Many thanks on behalf of the WWW central support team.
    <P><INPUT TYPE=submit> <INPUT TYPE=reset>
</FORM>
</body>
```

Browser 화면 및 Key-In 내용

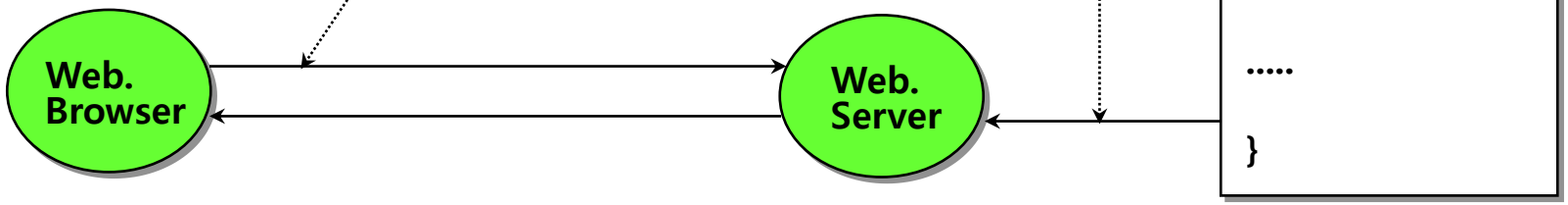


Web Browser가 Server에게 보내는 message

```
POST /cgi-bin/post_query HTTP/1.0
Accept: www/source
Accept: text/html
Accept: video/mpeg
Accept: image/jpeg
Accept: image/x-tiff
Accept: image/x-rgb
Accept: image/x-xbm
Accept: image/gif
Accept: application/postscript
User-Agent:  Lynx/2.2  libwww/2.14
From:  jhnang@plsun.sogang.ac.kr
Content-type: application/x-www-form-urlencoded
Content-length: 150
     * a blank line *
org=Sogang%20University
&users=30
&browsers=mosaic
&others=Netscape%20Explorer
&contact=jhnang@ccs.sogang.ac.kr
```

Web Server가 Browser에게 보내는 응답 message

```
Content-type: text/html
     * a blank line *
<H1>Query Results</H1>
You submitted the following name/value pairs:
<ul>
<li>org = Sogang University
<li>users = 30
<li>browsers = xmosaic
<li>others = Netscape Explorer
<li>contact = jhnang@ccs.sogang.ac.kr
</ul>
```



~jhnang/cgi-bin/post_query.cgi

• 상태 정보 유지 모델이 필요한 이유

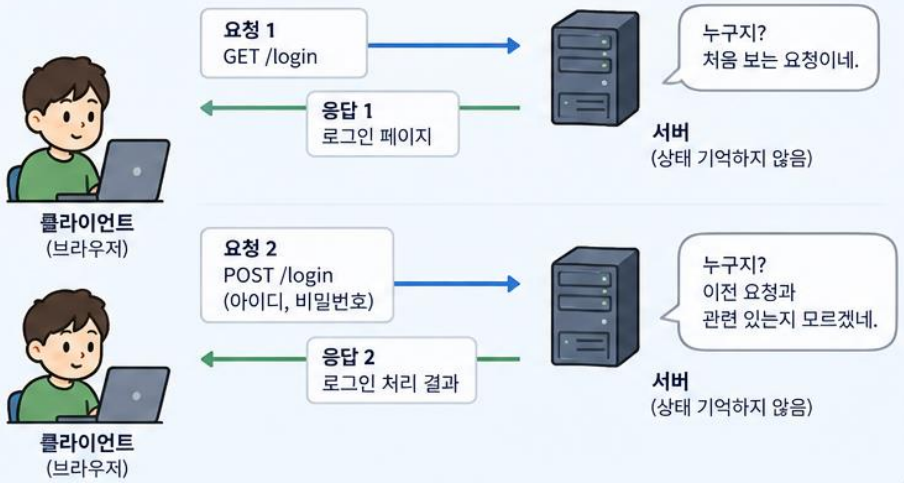
HTTP는 요청-응답 방식의 무상태(Stateless) 프로토콜입니다.

각 요청은 서로 독립적이어서, 서버는 이전 요청에 대한 정보를 기억하지 않습니다.

그래서 상태 유지가 필요한 이유는 무엇일까요?

1 HTTP의 기본 동작 (무상태)

각 요청은 독립적이어서 서버는 이전 요청을 기억하지 않습니다.



2 상태 유지가 필요한 이유

실제 웹 서비스는 여러 번의 요청으로 이루어진 연속적인 작업을 수행합니다.
이 때 사용자를 식별하고, 이전 요청과의 연관성을 유지해야 합니다.



1. 로그인 상태 유지

매번 아이디와 비밀번호를 입력하지 않고, 로그인 상태를 유지하여 사용자를 식별해야 합니다.

한 번 로그인하면 계속 이용하고 싶어요!'



2. 장바구니 관리

여러 페이지를 돌러봐도 선택한 상품이 장바구니에 계속 남아 있어야 합니다.

다른 페이지를 봐도 내가 담은 상품이 유지되어야 해요!'



3. 개인화된 서비스 제공

사용자의 설정, 최근 본 상품, 언어, 매마 등을 기억하여 더 편리한 경험을 제공해야 합니다.

내가 설정한 언어와 태마가 계속 유지되면 좋겠어요!



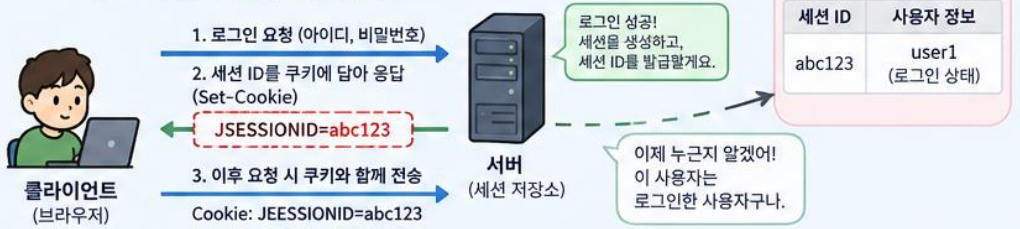
4. 보안 및 권한 관리

로그인한 사용자의 권한을 확인하고, 민감한 작업(예: 결제, 계좌 조회)에 대해 안전하게 접근을 제어해야 합니다.

내가 로그인한 상태에서만 결제를 할 수 있어야 해요!

3 상태 유지의 예시: 세션과 쿠키

서버는 쿠키(session id)나 세션을 이용해 사용자를 식별하고, 이전 요청들과의 연관성을 유지합니다.



4 상태가 없으면 생기는 문제

상태를 유지할 수 없다면, 사용자 경험과 보안, 서비스 기능에 많은 문제가 발생합니다.

구분	발생하는 문제	예시
사용자 식별 불가	매 요청마다 사용자를 알 수 없음	매번 로그인해야 함
연속 작업 불가	이전 단계와의 연관성을 알 수 없음	장바구니, 주문 과정이 매번 초기화됨
개인화 서비스 제공 불가	사용자 설정을 기억할 수 없음	언어, 태마, 최근 본 상품 등 유지 불가
보안 및 권한 관리 어려움	로그인 상태와 권한을 확인할 수 없음	로그인한 사용자만 접근해야 하는 페이지도 누구나 접근 가능



따라서, HTTP가 무상태 프로토콜이더라도, 실제 웹 서비스에서는 **사용자 식별, 연속 작업 처리, 개인화, 보안** 등을 위해 **세션/쿠키를 이용한 상태 유지가 필수적**입니다.



- **방법 1 : Hidden Tag 사용**

Hidden Tag(숨은 필드)를 이용한 상태 유지 방법

HTTP는 기본적으로 상태를 기억하지 않지만, 웹 페이지의 숨은 필드(hidden input)에 상태 정보를 담아 다음 요청과 함께 전송함으로써 사용자의 이전 상태를 유지할 수 있습니다.



1 사용자가 로그인

사용자가 아이디와 비밀번호를 입력하여 로그인합니다.

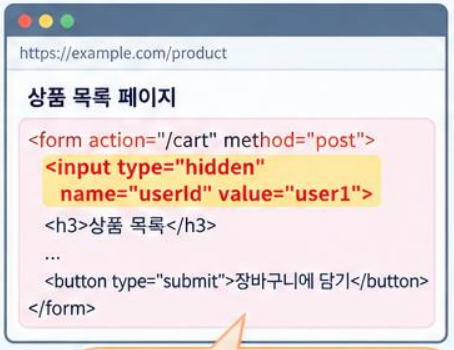




사용자
(브라우저)

2 서버가 상태 정보와 함께 다음 페이지를 응답

로그인이 성공하면, 서버는 사용자를 식별할 수 있는 정보를 숨은 필드(hidden input)에 담아 다음 페이지를 보냅니다.



눈에는 보이지 않지만,
userId가 페이지 안에 포함되어 있습니다.
(사용자가 페이지 소스를 보면 확인 가능)

3 사용자의 다음 요청 시 숨은 필드가 함께 전송

사용자가 상품을 장바구니에 담거나 다른 기능을 요청하면, 폼이 전송될 때 hidden 필드의 값도 함께 서버로 보송됩니다.



숨은 필드에 있던 userId 값이
요청 파라미터로 함께 전송됩니다.



사용자
(브라우저)

4 서버가 전송된 값으로 사용자를 식별하여 상태 유지

서버는 전달된 userId 값을 이용하여 이전 로그인 상태를 식별하고, 해당 사용자의 요청으로 처리합니다.



웹 서버

userId=user1 이니까
로그인한 사용자로
처리하자!

이후에도 모든 페이지에
숨은 필드를 포함시켜, 동일한 값을
계속 전송하면 상태를 유지할 수
있습니다.

주요 특징

- <input type="hidden"> 태그를 사용하여 상태 정보를 페이지에 포함시킵니다.
- 사용자가 다음 요청을 보낼 때, 이 값이 폼 데이터로 함께 전송됩니다.
- 서버는 전송된 값을 이용해 사용자를 식별하고 상태를 유지합니다.

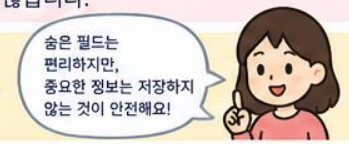
장점

- 구현이 간단합니다.
- 별도의 서버 저장소(세션 저장소 등)가 필요하지 않습니다.
- 소규모 웹 애플리케이션에서 간단한 상태 유지에 활용할 수 있습니다.

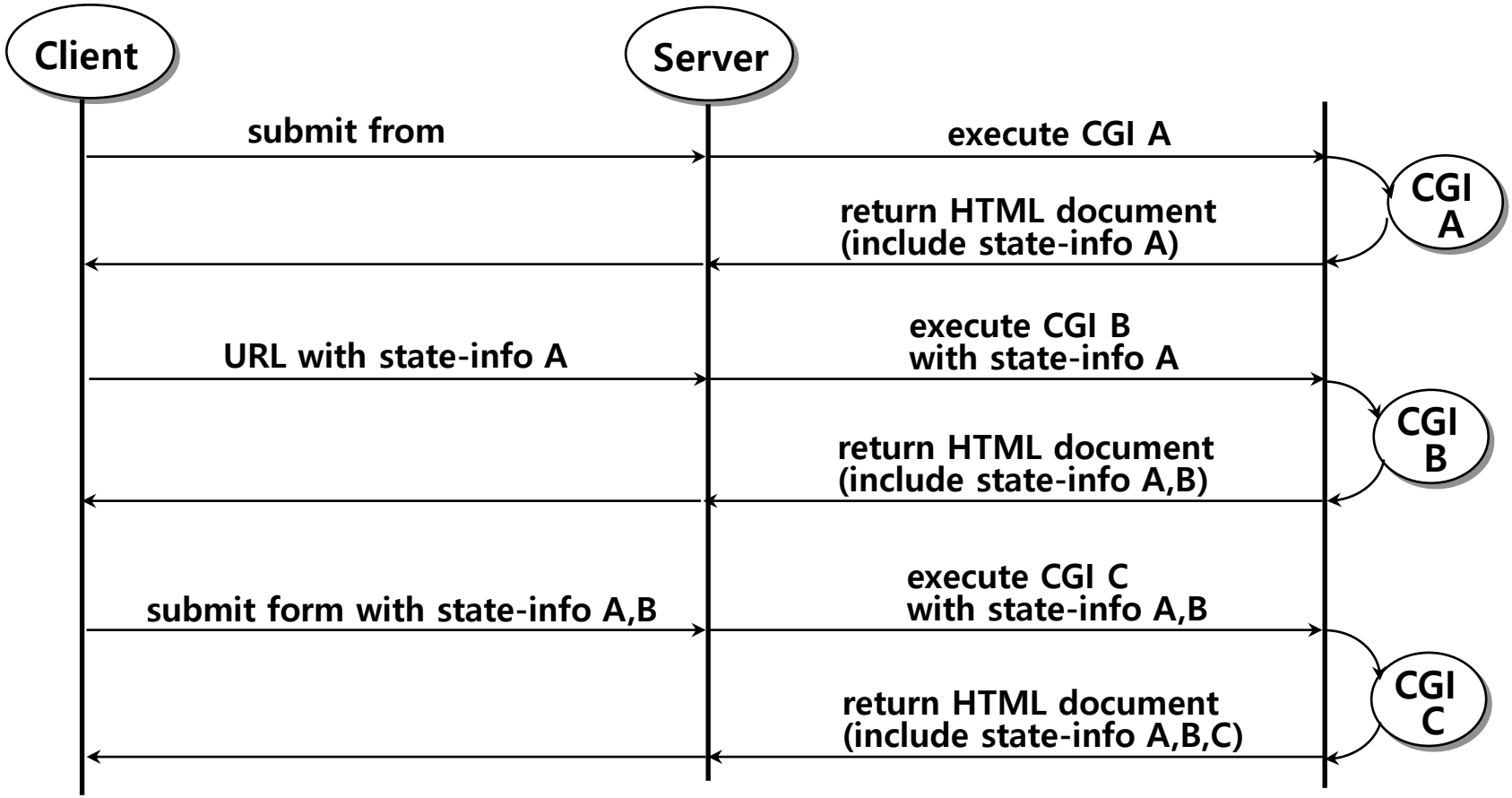
단점 및 주의사항

- 값이 클라이언트(브라우저)에 노출되므로, 사용자가 임의로 수정할 수 있습니다. (보안 취약)
- 중요한 정보(비밀번호, 권한 정보 동)는 저장하면 안 됩니다.
- 페이지마다 hidden 필드를 포함해야 하므로, 누락 시 상태가 유지되지 않습니다.

Hidden Tag는 **간단한 상태 유지 방법이지만, 보안성이 낮기** 때문에 실제 서비스에서는 주로 쿠키나 세션을 함께 사용합니다.



– CGI들의 진행 과정 (CGI들사이에 State를 유지하면서 수행)



• 방법 2 : Cookie 사용

쿠키(Cookie)를 이용한 상태 유지의 원리

HTTP는 기본적으로 stateless(무상태) 있지만, 쿠키를 사용하면 사용자를 식별하여 상태를 유지할 수 있습니다.

1 사용자가 로그인 요청

사용자가 아이디와 비밀번호를 입력하여 로그인합니다.



2 서버가 쿠키를 생성하여 응답

서버는 로그인 정보를 확인한 후, 사용자를 식별할 수 있는 값(예: 세션 ID)을 쿠키에 담아 브라우저로 돌려줍니다.



3 이후 요청 시 쿠키를 함께 전송

사용자가 다른 페이지를 요청할 때, 브라우저는 저장해둔 쿠키를 자동으로 함께 전송합니다.



4 서버가 쿠키로 사용자를 식별하여 상태 유지

서버는 전달받은 쿠키의 세션 ID를 통해 사용자가 이전에 로그인한 사용자임을 확인하고, 로그인된 상태로 서비스를 제공합니다.



쿠키 덕분에, HTTP의 무상태 특성에서도 사용자의 상태(로그인 여부 등)를 유지할 수 있습니다!

쿠키는 브라우저에 저장되는 작은 데이터로, 매 요청마다 함께 전송되어 서버가 사용자를 식별할 수 있게 해줍니다.



- Syntax of the Set-Cookie **HTTP Response Header**

```
Set-Cookie: NAME=VALUE; expires=DATE; path=PATH; domain=DOMAIN_NAME; secure
```

- Syntax of the Cookie **HTTP Request Header**

```
Cookie: NAME1=OPAQUE_STRING1; NAME2=OPAQUE_STRING2 ...
```

- Example

Client requests a document, and receives in the response:

```
Set-Cookie: CUSTOMER=WILE_E_COYOTE; path=/; expires=Wednesday, 09-Nov-99
```

When client requests a URL in path "/" on this server, it sends:

```
Cookie: CUSTOMER=WILE_E_COYOTE
```

Client requests a document, and receives in the response:

```
Set-Cookie: PART_NUMBER=ROCKET_LAUNCHER_0001; path=/
```

When client requests a URL in path "/" on this server, it sends:

```
Cookie: CUSTOMER=WILE_E_COYOTE; PART_NUMBER=ROCKET_LAUNCHER_0001
```

Client receives:

```
Set-Cookie: SHIPPING=FEDEX; path=/foo
```

When client requests a URL in path "/" on this server, it sends:

```
Cookie: CUSTOMER=WILE_E_COYOTE; PART_NUMBER=ROCKET_LAUNCHER_0001
```

Static scoping rule(?)

When client requests a URL in path "/foo" on this server, it sends:

```
Cookie: CUSTOMER=WILE_E_COYOTE; PART_NUMBER=ROCKET_LAUNCHER_0001; SHIPPING=FEDEX
```

PHP Source:

```
<!DOCTYPE html>
<?php
$cookie_name = "user";
$cookie_value = "John Doe";
setcookie($cookie_name, $cookie_value, time() + (86400 * 30), "/"); // 86400 = 1
day
?>
<html>
<body>

<?php
if(!isset($_COOKIE[$cookie_name])) {
      echo "Cookie named '" . $cookie_name . "' is not set!";
} else {
      echo "Cookie '" . $cookie_name . "' is set!<br>";
      echo "Value is: " . $_COOKIE[$cookie_name];
}
?>

<p><strong>Note:</strong> You might have to reload the page to see the value of
the cookie.</p>

</body>
</html>
```

브라우저마다 쿠키를 저장하는 위치와 파일이 다릅니다.

각 브라우저는 보안을 위해 서로 다른 폴더와 형식의 파일에 쿠키를 저장합니다. 파일 안에는 아래와 같은 주요 정보가 담겨 있습니다.

Google Chrome

쿠키 저장 위치 예시

Windows

C:\Users\사용자명\AppData\Local\Google\Chrome\User Data\Default\

macOS

~/Library/Application Support/Google/Chrome/Default/

Linux

~/.config/google-chrome/Default/

쿠키 파일

Cookies
(SQLite 데이터베이스 파일)

모든 쿠키정보가 이 데이터베이스 파일에 저장됩니다.

Microsoft Edge

쿠키 저장 위치 예시

Windows

C:\Users\사용자명\AppData\Local\Microsoft\Edge\User Data\Default\

macOS

~/Library/Application Support/Microsoft Edge/Default/

Linux

~/.config/microsoft-edge/Default/

쿠키 파일

Cookies
(SQLite 데이터베이스 파일)

Chrome과 유사한 구조의 데이터베이스 파일을 사용합니다.

Mozilla Firefox

쿠키 저장 위치 예시

Windows

C:\Users\사용자명\AppData\Roaming\Mozilla\Firefox\Profiles\<프로필폴더>\

macOS

~/Library/Application Support/Firefox/Profiles/<프로필폴더>/

Linux

~/.mozilla/firefox/<프로필폴더>/

쿠키 파일

cookies.sqlite
(SQLite 데이터베이스 파일)

프로필 폴더마다 별도의 쿠키 데이터베이스 파일이 존재합니다.

Apple Safari

쿠키 저장 위치 예시

macOS

~/Library/Cookies/

쿠키 파일

Cookies.binarycookies
(바이너리 형식)

바이너리 형식의 자체 포맷으로 쿠키를 저장합니다.

Safari는 macOS와 iOS에서 자체 형식의 바이너리 파일을 사용하며, 다른 브라우저와 파일 형식이 다릅니다.

쿠키 파일에 저장되는 주요 정보 (예시)

브라우저의 쿠키 파일(데이터베이스)에는 보통 다음과 같은 정보가 저장됩니다.

이름	값	도메인	경로	만료 시간	보안 설정
sessionid	abc123...	www.example.com	/	2025-12-31 23:59:59	Secure, HttpOnly, SameSite=Lax

※ 브라우저마다 내부 구조나 필드 이름은 조금씩 다를 수 있지만, 기본적으로 위와 같은 정보가 저장됩니다.

주요 정보 설명

이름 (Name)	쿠키의 이름 (예: sessionid)
값 (Value)	서버가 저장한 값 (예: abc123...)
도메인 (Domain)	이 쿠키가 전송될 도메인 (예: www.example.com)
경로 (Path)	이 쿠키가 유효한 경로 (예: / 또는 /mypage)
만료 시간 (Expires/Max-Age)	쿠키가 만료되는 시점 (세션 쿠키는 만료 시간이 없음)
보안 설정	Secure(HTTPS에서만 전송), **HttpOnly**(자바스크립트 접근 불가), SameSite(CSRF 방지 정책) 등



따라서, 다른 브라우저의 쿠키 파일은 직접 열어도 서로 호환되지 않습니다.

브라우저별로 저장 위치, 파일 형식, 내부 구조가 다르기 때문에, 한 브라우저의 쿠키를 다른 브라우저에서 그대로 사용할 수 없습니다.

ChatGPT는 어떻게 이전 대화 내용을 기억하고 활용할까?

ChatGPT는 대화의 상태(state)를 '대화 맥락(context)'으로 유지하여, 매번 이 맥락을 함께 모델에 입력함으로써 이전 질문과 답변을 기억하고 일관성 있게 답변할 수 있습니다.

1 사용자의 입력

사용자가 새로운 질문을 입력합니다.



2 대화 맥락 구성

서버는 현재 입력과 이전 대화 내용을 모아 하나의 "프롬프트(context)"로 만듭니다.

프롬프트(모델에 전달되는 내용) 예시

system	당신은 도움이 되는 AI 어시스턴트입니다. ...
user	Web 서버에서 PHP 프로그램이 수행되는 방법을 설명해줘.
assistant	(이전에 생성한 답변 ...)
⋮	
user	CGI 프로그램도 웹 서버를 통해 결과가 전달되나요?

3 언어 모델(LLM)이 응답 생성

구성된 전체 대화 맥락을 모델에 입력하여, 맥락을 이해한 상태에서 다음 답변을 생성합니다.

LLM
(예: GPT-4)

모델은 입력으로 주어진 전체 대화 내용을 보고, 이전 질문과 답변을 이해한 상태에서 일관된 답변을 만듭니다.

4 응답 반환

생성된 답변을 사용자에게 전달하고, 이 답변도 대화 맥락에 추가됩니다.

네, 일반적인 CGI 프로그램(예: C로 작성한 CGI 프로그램)도 웹 서버를 통해 실행되고, 그 출력 결과는 웹 서버가 받아서 HTTP 응답으로 브라우저에 전송합니다. ...

5 대화 맥락 업데이트

방금 주고받은 입력과 출력을 대화 기록에 추가하여, 다음 질문에서도 계속 활용할 수 있습니다.

대화 기록 (서버 측)

user	...
assistant	...
user	CGI 프로그램도 웹 서버를 통해 ...
assistant	네, 일반적인 CGI 프로그램도 ...
⋮	

대화 기록의 저장 위치

대화 내용은 사용자의 세션에 연결되어 OpenAI의 서버에 저장됩니다.

일시적인 컨텍스트 (모델 입력용)

- 현재 대화 세션의 최근 여러 메시지를 매 요청마다 모델에 함께 전달합니다.
- 모델에는 한 번에 보낼 수 있는 최대 길이(컨텍스트 윈도우, 예: 수십만 토큰)가 있어, 너무 긴 대화는 오래된 부분이 자동으로 제외될 수 있습니다.

지속적인 대화 기록 (계정에 저장)

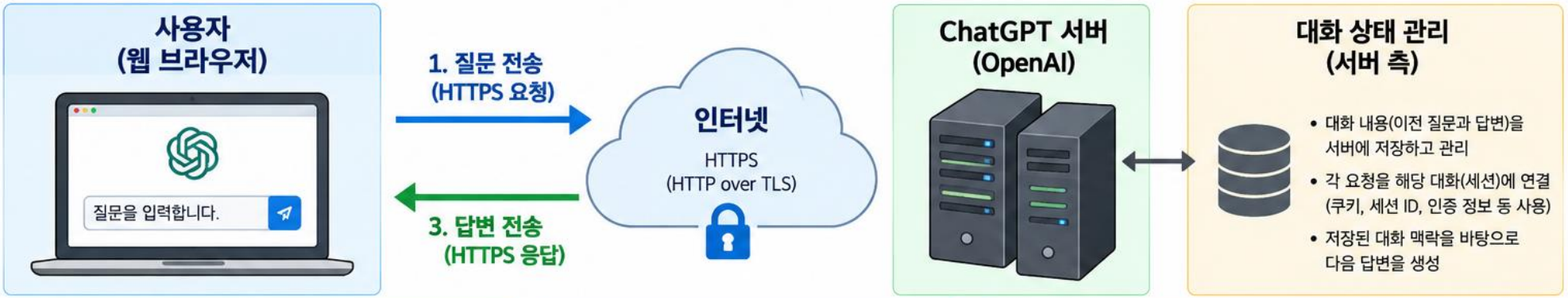
- 로그인한 사용자의 경우, 대화 내용이 계정에 저장되어 이후에도 이어서 대화할 수 있습니다.
- 사용자가 대화 기록을 삭제하면 더 이상 해당 기록은 사용되지 않습니다.
- 저장 기간 및 정책은 OpenAI의 개인정보 처리방침에 따릅니다.

핵심 정리

- ChatGPT의 "state"는 별도의 변수로 저장하는 것이 아니라, 이전 대화 내용을 포함한 전체 대화 맥락을 매번 모델에 입력하는 방식으로 유지됩니다.
- 이 덕분에 모델이 이전 질문과 답변을 이해하고, 일관성 있는 답변을 생성할 수 있습니다.
- 다만, 컨텍스트 윈도우의 한계로 매우 긴 대화의 일부는 자동으로 제외될 수 있으며, 대화 기록은 사용자의 계정에 저장 · 관리됩니다.

ChatGPT와의 대화는 어떻게 전송되고, 상태는 어떻게 유지될까?

사용자의 질문과 ChatGPT의 답변은 HTTPS(HTTP over TLS)로 전송되며, 대화의 상태는 서버 측에서 관리됩니다.



1. 사용자의 질문 (HTTPS 요청 예시)

```
POST /v1/chat/completions HTTP/1.1
Host: api.openai.com
Authorization: Bearer sk-...
Content-Type: application/json
Cookie: session=abc123...

{
   "model": "gpt-4o",
   "messages": [
     {"role": "user", "content": "CGI 프로그램도
                웹 서버를 통해 결과가 전달되나요?"}
   ],
   "stream": true
}
```

질문 내용이 HTTPS로 서버에 전송됩니다. (실제로는 더 많은 필드가 포함될 수 있음)

2. 서버에서의 처리

1. 요청 수신 (HTTPS)
2. 인증 및 대화 세션 확인 (쿠키, 세션 ID 등으로 이전 대화 조회)
3. 이전 대화 내용을 포함하여 언어 모델(LLM)에 전달
4. 생성된 답변을 스트리밍 응답으로 전송

3. ChatGPT의 답변 (스트리밍 HTTPS 응답 예시)

```
HTTP/1.1 200 OK
Content-Type: text/event-stream
Transfer-Encoding: chunked

data: {"choices":[{"delta":{"content":"'네."}}]}\n\n

data: {"choices":[{"delta":{"content":"' 일반적으로""}}]}\n\n

data: {"choices":[{"delta":{"content":"..."}}]}\n\n

data: {"choices":[{"delta":{"content":"\n\n"}}]}\n\n

data: [DONE]\n\nn
```

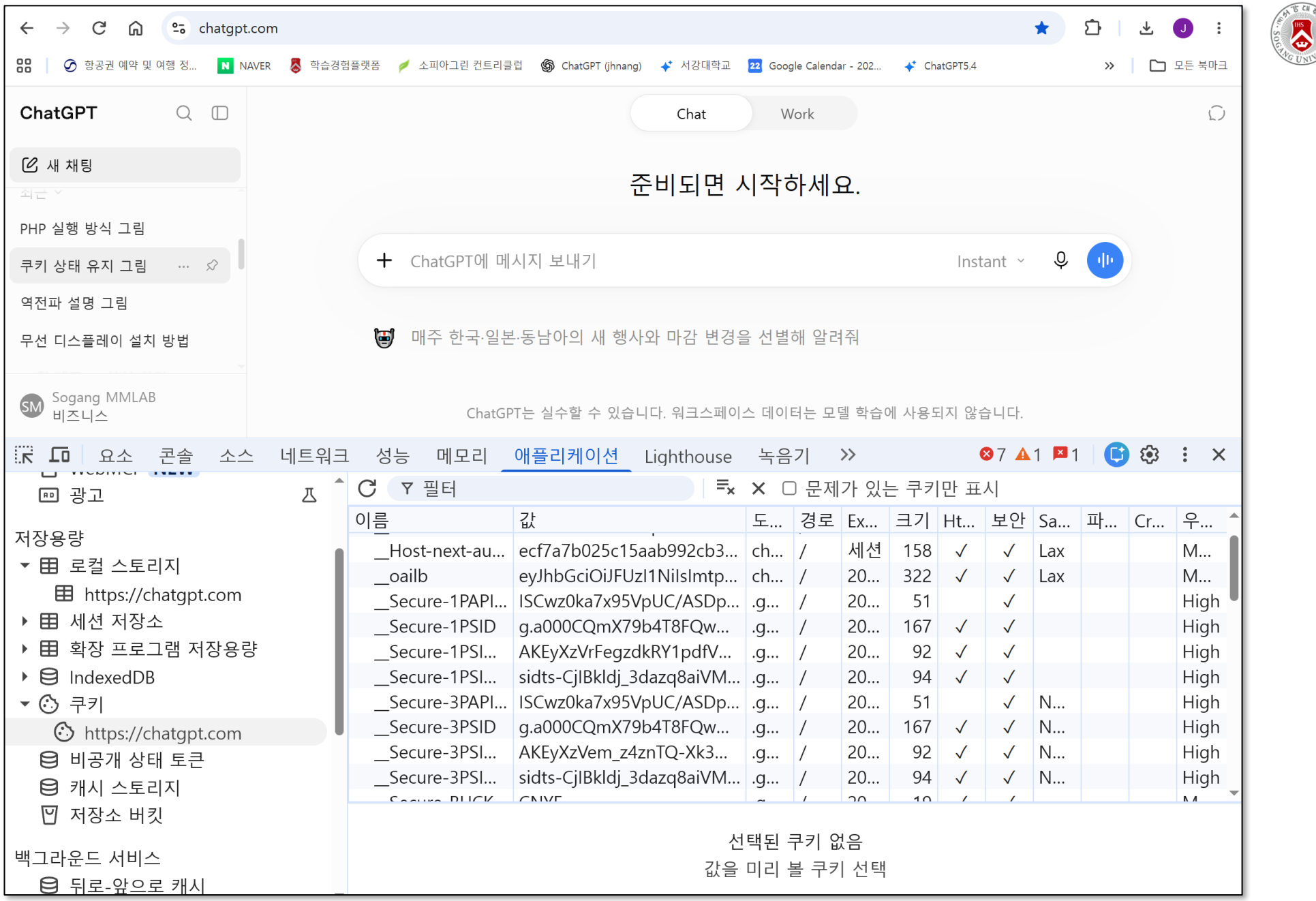
답변이 조각(chunk) 단위로 계속 전송되어, 브라우저에 글자가 순차적으로 표시됩니다.

HTTP는 Stateless, 상태 관리는 서버에서

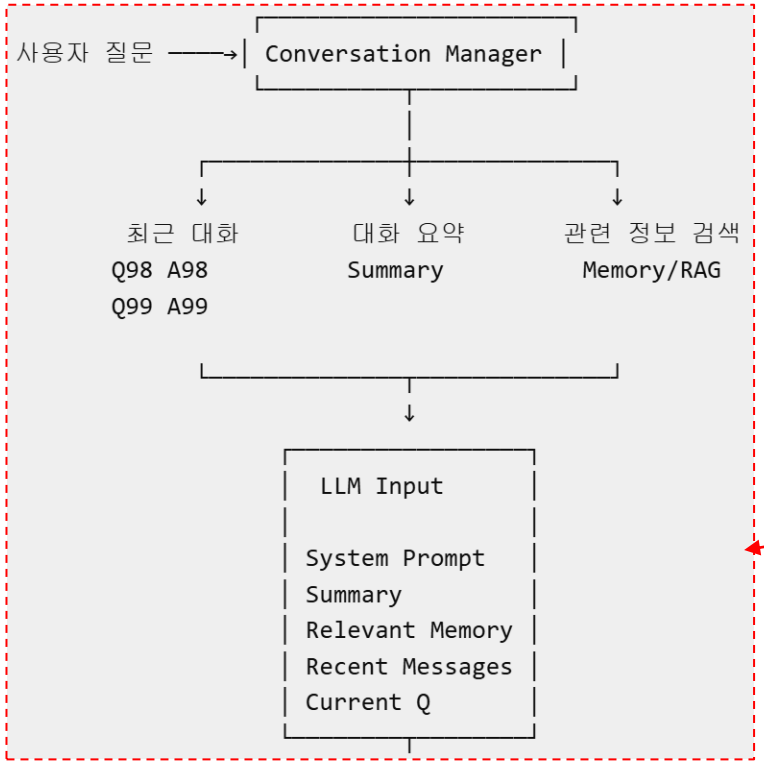
- HTTP 프로토콜은 기본적으로 상태를 저장하지 않는 Stateless 프로토콜입니다.
- 즉, 이전 대화 내용을 HTTP가 자동으로 기억하지 않습니다.
- ChatGPT는 서버 측에서 대화 내용을 저장하고, 각 요청을 올바른 대화와 연결하여 이전 맥락을 반영한 답변을 생성합니다.

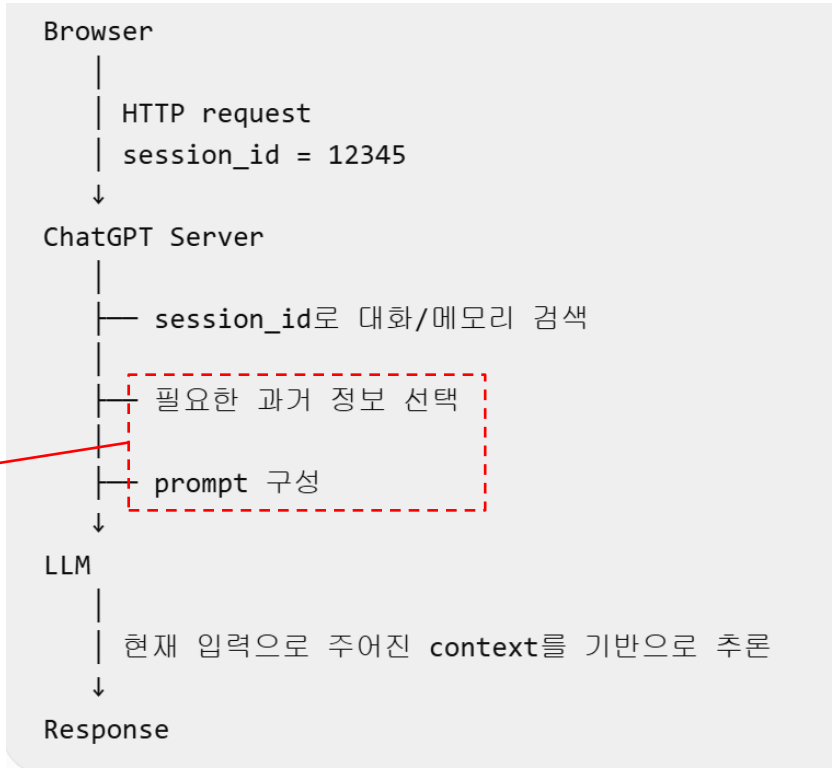
요약

1. 질문과 답변은 HTTPS(HTTP over TLS)로 전송됩니다.
2. 답변은 보통 스트리밍 방식으로 전달되어 실시간으로 표시됩니다.
3. 대화의 상태(이전 질문과 답변)는 서버 측에서 관리되며, HTTP 자체가 기억하지 않습니다.
4. 브라우저는 단순히 HTTP 요청을 보내고, 서버가 생성한 응답을 받아 화면에 표시합니다.


chatgpt.com
ChatGPT
새 채팅
PHP 실행 방식 그림
쿠키 상태 유지 그림
역전파 설명 그림
무선 디스플레이 설치 방법
Sogang MMLAB
비즈니스
Chat
Work
준비되면 시작하세요.
ChatGPT에 메시지 보내기
Instant
매주 한국·일본·동남아의 새 행사와 마감 변경을 선별해 알려줘
ChatGPT는 실수할 수 있습니다. 워크스페이스 데이터는 모델 학습에 사용되지 않습니다.
요소
콘솔
소스
네트워크
성능
메모리
애플리케이션
Lighthouse
녹음기
광고
저장용량
로컬 스토리지
https://chatgpt.com
세션 저장소
확장 프로그램 저장용량
IndexedDB
쿠키
https://chatgpt.com
비공개 상태 토큰
캐시 스토리지
저장소 버킷
백그라운드 서비스
뒤로-앞으로 캐시
필터
문제가 있는 쿠키만 표시
이름 값 도... 경로 Ex... 크기 Ht... 보안 Sa... 파... Cr... 우...
__Host-next-au... ecf7a7b025c15aab992cb3... ch... / 세션 158 ✓ ✓ Lax M...
__oailb eyJhbGciOiJFUzI1NiIsImtp... ch... / 20... 322 ✓ ✓ Lax M...
__Secure-1PAPI... ISCwz0ka7x95VpUC/ASDp... .g... / 20... 51 ✓ High
__Secure-1PSID g.a000CQmX79b4T8FQw... .g... / 20... 167 ✓ ✓ High
__Secure-1PSI... AKEyXzVrFegzdkRY1pdfV... .g... / 20... 92 ✓ ✓ High
__Secure-1PSI... sidts-CjIBkIdj_3dazq8aiVM... .g... / 20... 94 ✓ ✓ High
__Secure-3PAPI... ISCwz0ka7x95VpUC/ASDp... .g... / 20... 51 ✓ N... High
__Secure-3PSID g.a000CQmX79b4T8FQw... .g... / 20... 167 ✓ ✓ N... High
__Secure-3PSI... AKEyXzVem_z4znTQ-Xk3... .g... / 20... 92 ✓ ✓ N... High
__Secure-3PSI... sidts-CjIBkIdj_3dazq8aiVM... .g... / 20... 94 ✓ ✓ N... High
선택된 쿠키 없음
값을 미리 볼 쿠키 선택

- chatgpt에서 session을 유지하기 위하여 그 전 질문 및 답변을 LLM 입력 프롬프트에 계속 append하면 나중에는 LLM에 입력되는 프롬프트가 너무 길어져서 비용이 점점 증가하게 되는 것이 아닌가?
  - 해결 방법 : Context window 제한, 대화 요약(summary), 필요한 과거 정보만 검색(retrieval), 별도의 상태 정보 유지





1번째 요청:
[System Prompt]
Q1

2번째 요청:
[System Prompt]
Q1
A1
Q2

3번째 요청:
[System Prompt]
Q1
A1
Q2
A2
Q3

...

100번째 요청:
[System Prompt]
Q1 A1
Q2 A2
...
Q99 A99
Q100

*** ChatGPT가 이전 대화를 기억한다**
- 웹/서버 계층: 세션, 사용자, 대화 기록 등을 관리
- LLM 계층: 서버가 이번 요청에 제공한 context를 읽고 답변 생성

ChatGPT의 대화 처리 전체 구조

(HTTP Session → 대화 저장 → 메모리/요약/검색 → Context 구성 → LLM)

1 사용자 (브라우저/앱)

이번에 session에 대해 정리해줘.



HTTP Request

```
POST /chat
Cookie: session_id=abc123
{
  "message": "이번에 session에 대해 정리해줘."
}
```

2 웹/서버 계층 (Conversation Manager)



- 세션 식별 (session_id)
- 사용자 인증
- 대화 히스토리 조회
- 관련 정보 검색 (RAG)
- 대화 요약 생성/업데이트
- 프롬프트 구성
- LLM API 호출
- 응답 후 대화 저장

3 데이터 저장소

대화 기록 DB
(전체 히스토리 저장)

Q1 ...
A1 ...
Q2 ...
A2 ...
...

요약 메모리
(장기 요약 정보)

- 사용자 관심사
- 주요 주제
- 중요한 사실
- 대화 요약 ...

외부 지식/문서
(RAG, 선택적)

- 사용자 업로드 문서
- 사내 문서
- 웹 검색 결과
- ...

4 컨텍스트 구성 (프롬프트 빌드)

- **시스템 프롬프트** (역할, 지침 등)
- **대화 요약** (오래된 내용 압축)
- **관련 과거 대화** (최근 N개, 선택적)
- **관련 검색 결과** (RAG, 선택적)
- **현재 사용자 질문**

구성된 프롬프트 입력

5 LLM (예: GPT-4)



컨텍스트 윈도우
(예: 128K tokens)

- 시스템 프롬프트 토큰
- 요약 토큰
- 과거 대화 토큰
- 검색 결과 토큰
- 현재 질문 토큰

답변 생성

응답 (텍스트)

응답을 사용자에게 반환 (HTTP Response)

✕ 왜 모든 대화를 계속 넣지 않을까?

- 대화가 길어질수록 입력 토큰 수 증가
- 비용 증가 (입력 토큰 요금)
- 처리 시간 증가 (응답 지연)
- 모델의 context window 한계 (예: 128K)
- 불필요한 정보로 성능 저하 가능

✓ 대신 이렇게 처리한다!

1. 최근 대화만 포함 (예: 최근 N개 메시지)
2. 오래된 대화는 요약하여 압축
3. 현재 질문과 관련된 과거 정보만 검색 (RAG)
4. 사용자 선호, 중요한 사실 등은 별도 메모리로 관리
5. 세션 정보는 서버에서 관리하고, LLM에는 필요한 컨텍스트만 전달

컨텍스트 구성 예시

```
[System Prompt]
당신은 도움이 되는 AI 어시스턴트입니다. ...

[Conversation Summary]
사용자는 PHP/CGI와 HTTP의 동작을 공부하고 있다.
- HTTP는 request-response 방식임
- ... (요약 내용)

[Recent Conversation]
Q98: ...
A98: ...
Q99: ...
A99: ...

[Retrieved Information]
(관련 문서 내용 ...)

[Current Question]
Q100: 이번에 session에 대해 정리해줘.
```

토큰 수와 비용의 차이 (개념적 예시)

대화 방식	업력 토큰 수 (예시)	비용 (상대적)
모든 대화 계속 추가	100,000	매우 큼
최근 10개만 사용	10,000	보통
요약 + 최근 대화	4,000	작음
요약 + 검색(RAG)	2,000	더 작음

핵심: 어떤 과거 정보를 LLM에 넣을 것인가가 비용과 성능을 좌우합니다.

5. 프로그래밍 과제

- C를 이용한 CGI program 작성
  - "/tmp/jhnang/public_html/cgi-bin/post_query.c" 프로그램을 "GET" method를 사용하는 CGI프로그램으로 수정하여라.
  - 호출하는 프로그램은 아래와 같다.

CGI 프로그램 수행 요청

HTML Source http://cspro.sogang.ac.kr/~jhnang/CGI_RequestExample.html

```
<html>
<head>
<title>This is a practice form.</title>
<link rev=made href="mailto:grobe@kuhub.cc.ukans.edu">
</head>
<body>
<FORM METHOD=GET
      ACTION="http://cspro.sogang.ac.kr/~jhnang/cgi-bin/post_query.cgi">
Please help us to improve the World Wide Web by filling in the
following questionaire:
   <P>Your organization? <INPUT NAME="org" TYPE=text SIZE="48">
   <P>Commercial? <INPUT NAME="commerce" TYPE=checkbox>
        How many users? <INPUT NAME="users" TYPE=int>
   <P>Which browsers do you use?
   <OL>
    <LI>Cello <INPUT NAME="browsers" TYPE=checkbox VALUE="cello">
    <LI>Lynx <INPUT NAME="browsers" TYPE=checkbox VALUE="lynx">
    <LI>X Mosaic <INPUT NAME="browsers" TYPE=checkbox VALUE="mosaic">
    <LI>Others <INPUT NAME="others" SIZE=40>
   </OL>
A contact point for your site: <INPUT NAME="contact" SIZE="42">
<P>Many thanks on behalf of the WWW central support team.
   <P><INPUT TYPE=submit> <INPUT TYPE=reset>
</FORM>
</body>
```

수정할 부분

C 언어를 이용한 CGI 프로그램

```
#include <stdio.h>
#define MAX_ENTRIES 10000
typedef struct {
    char *name;
    char *val;
} entry;
char *getenv();
char *makeword(char *line, char stop);
char *fmakeword(FILE *f, char stop, int *len);
char x2c(char *what);
void unescape_url(char *url);
void plustospace(char *str);

main(int argc, char *argv[]) {
    entry entries[MAX_ENTRIES];
    register int x,m=0;
    int cl;
    printf("Content-type: text/html%c%c",10,10);
    if(strcmp(getenv("REQUEST_METHOD"),"POST")) {
        printf("This script should be referenced with a METHOD of POST.\n");
        exit(1);
    }
    if(strcmp(getenv("CONTENT_TYPE"),"application/x-www-form-urlencoded")) {
        printf("This script can only be used to decode form results. \n");
        exit(1);
    }
    cl = atoi(getenv("CONTENT_LENGTH"));
    for(x=0;cl && (!feof(stdin));x++) {
        m=x;
        entries[x].val = fmakeword(stdin,'&',&cl);
        plustospace(entries[x].val);
        unescape_url(entries[x].val);
        entries[x].name = makeword(entries[x].val,'=');
    }
    printf("<H1>Query Results</H1>");
    printf("You submitted the following name/value pairs:<p>%c",10);
    printf("<ul>%c",10);
    for(x=0; x <= m; x++)
    printf("<li> <code>%s = %s</code>%c",
            entries[x].name, entries[x].val,10);
    printf("</ul>%c",10);
}
```

http://cspro.sogang.ac.kr/~jhnang/post_query.c

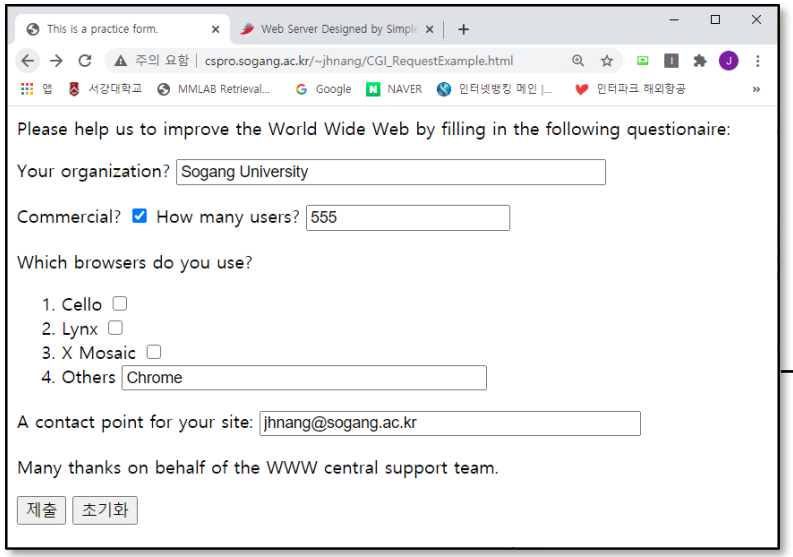
post_query_2019xxxx.c

```
% gcc -o aa.cgi
    post_query_2019xxxx.c
```

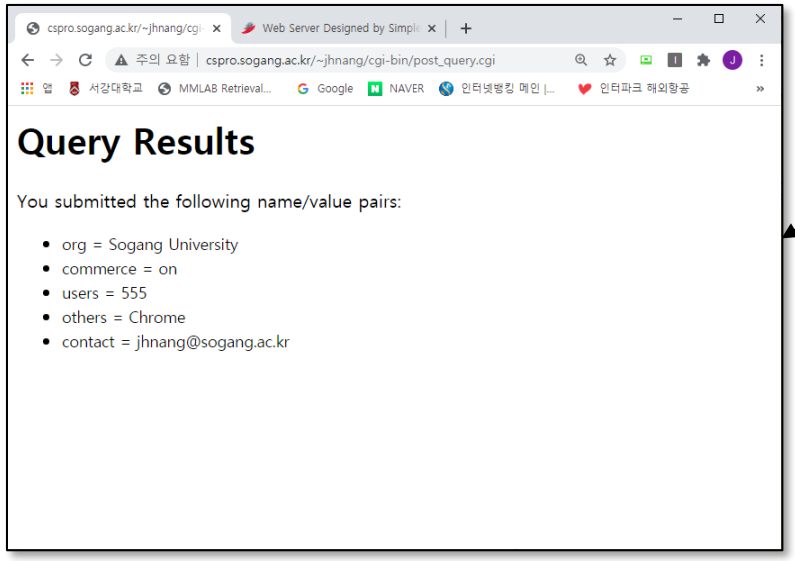
⇔자신의 "~user/public_html/cgi_bin/aa.cgi"로 제출한 C 파일을 컴파일하여 저장할 것

⇒ % chmod 755 aa.cgi

- 제출할 파일 (cyber.sogang.ac.kr)
  - ⇔ GET method를 위한 C언어 CGI 프로그램 소스
  - ⇔ 프로그램을 테스트할 URL (~user/public_html/Homework2_2019xxxx/html)



(~user/public_html/Homework2_2019xxxx.html



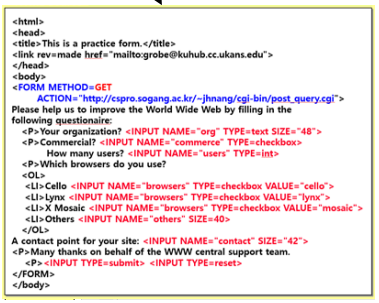
cspro.sogang.ac.kr

"~user/public_html/cgi_bin/aa.cgi

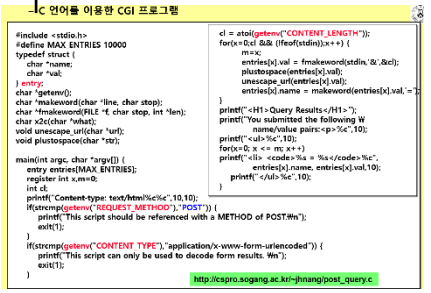
webserver APACHE

CGI 프로그램

% gcc -o aa.cgi post_query_2019xxxx.c



(~user/public_html/Homework2_2019xxxx/html



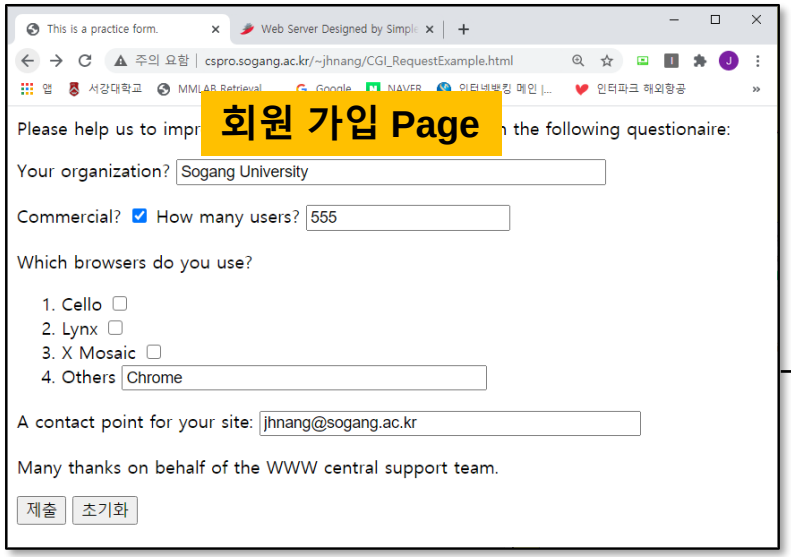
post_query_2019xxxx.c

- PHP 혹은 C를 이용한 CGI program 작성
  - 회원 가입을 받는 홈페이지 (HTML 문서)를 작성하고, 이를 받아서 항목별로 echo back하는 CGI 프로그램 (PHP 언어로 작성)을 작성하라.
    1) 가입 페이지에서 입력 받을 내용 (많을 수록 좋음)
       ① ID
       ② Password
       ③ 이메일
       ④ 주소
       ⑤ 학교
       ⑥ 학과
       ⑦ HP 번호
       ⑧ 관심 분야 (5개)
       ⑨ 기타

    과제 마감 : 10월 6일 (목) 오전 10:30
    제출 내용 : HTML 파일과 CGI 프로그램
    제출 장소 : Cyber Campus
    조교가 각 홈페이지를 직접 방문하여 동작여부 확인

    2) 입력 형식 : Text, Passwd, Check-box, Radio Button, Select (Choice), …

    3) 자신의 홈 디렉토리 (~cse20220111/public_html) 아래에 MemReg20220111.html로 저장
  - CGI 프로그램은 ~cse20220111/public_html/cgi-bin/에 저장



(~user/public_html/MemReg2022011.html

cspro.sogang.ac.kr

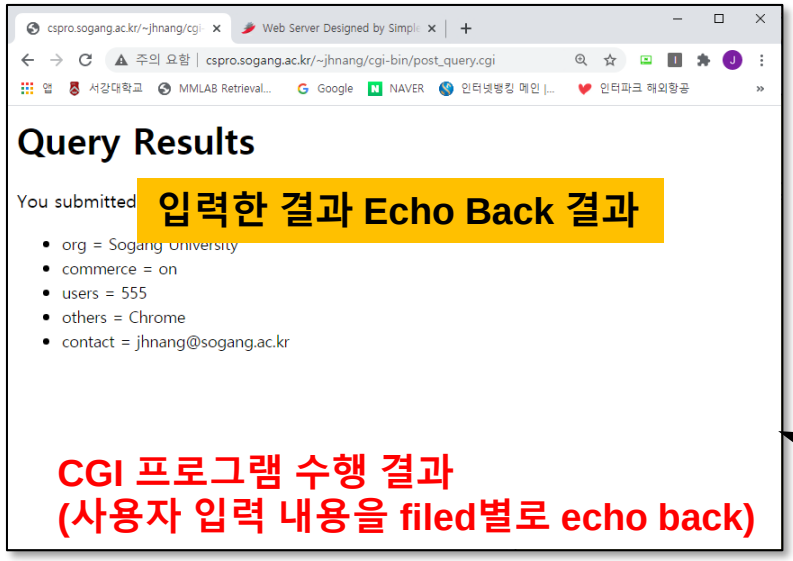
"~user/public_html/cgi_bin/aa.cgi

webserver APACHE

CGI 프로그램

```
<html>
<head>
<title>This is a practice form.</title>
<link rev=made href="mailto:grobe@kuhub.cc.ukans.edu">
</head>
<body>
<FORM METHOD=GET
      ACTION="http://cspro.sogang.ac.kr/~jhnang/cgi-bin/post_query.cgi">
Please help us to improve the World Wide Web by filling in the
following questionaire:
  <P>Your organization? <INPUT NAME="org" TYPE=text SIZE="48">
  <P>Commercial? <INPUT NAME="commerce" TYPE=checkbox>
       How many users? <INPUT NAME="users" TYPE=int>
  <P>Which browsers do you use?
  <OL>
   <LI>Cello <INPUT NAME="browsers" TYPE=checkbox VALUE="cello">
   <LI>Lynx <INPUT NAME="browsers" TYPE=checkbox VALUE="lynx">
   <LI>X Mosaic <INPUT NAME="browsers" TYPE=checkbox VALUE="mosaic">
   <LI>Others <INPUT NAME="others" SIZE=40>
  </OL>
A contact point for your site: <INPUT NAME="contact" SIZE="42">
<P>Many thanks on behalf of the WWW central support team.
    <P> <INPUT TYPE=submit> <INPUT TYPE=reset>
</FORM>
</body>
```

(~user/public_html/MemReg2022011.html



CGI 프로그램 수행 결과
(사용자 입력 내용을 filed별로 echo back)